



LAB REPORT- 01

Department of Computer Science & Engineering (CSE)

Faculty of Science and Information Technology (FSIT)

Spring 2026

Course Title : Operating Systems Lab

Course Code : CSE324

Course Credit : 1.5

Date of Submission : March 05 , 2026

Topic Covered : Linux Shell Scripting
(Experiment 01–13)

Submitted By

Shafin Ahmed

232-15-184

65_A1

Department of Computer Science & Engineering

Submitted To

Husne Mubarak

Lecturer

Department of Computer Science & Engineering

Faculty of FSIT

Comment By Course Teacher:

--

Table of Contents

Experiment No.	Experiment Name	Page No.
01	Read from Default Variable	01-05
02	Read Single Input Variable	06-09
03	Read from Default Variable (\$REPLY)	10-13
04	Read Multiple Inputs	14-17
05	Read Multiple Inputs Using Array	18-21
06	If Statement	22-26
07	While Loop	27-30
08	For Loop	31-34
09	Until Loop	35-38
10	Arithmetic Operations in Bash	39-46
11	Functions in Bash	47-50
12	Local Variables in Bash	51-54
13	Mini Project – Password Generator	55-58

Experiment No: 01

Experiment Name: Implementation of User Defined Variables in Bash Shell Script

Script Name:

user_defined_variable_demo.sh

Introduction

Overview

In this experiment, I have implemented a Bash shell script to understand the concept of user-defined variables in Linux. The primary objective of this experiment has been to learn how variables are declared, assigned, and accessed within a shell scripting environment. I have also examined common mistakes that occur during variable declaration.

Topics Covered

- User-defined variables in Bash
 - Variable assignment rules
 - Echo command usage
 - Variable expansion using `$`
 - Execution of shell script
-

Environment / Platform

I have performed this experiment using the Ubuntu Linux operating system with the Bash shell environment.

To create and edit the script file, I have used the graphical text editor **gedit**. I opened the script file using the following command:

```
gedit user_defined_variable_demo.sh
```

After writing the script inside the editor, I saved the file. Since I executed the script using the Bash interpreter directly, I did not need to modify file execution permissions.

Task

Description

The task was to create a Bash script that demonstrates how user-defined variables work. The script also intentionally included incorrect variable declarations to observe how Bash handles syntax errors.

Source Code

```
#!/bin/bash

echo "Hello World"

n=abcd
10val=10
echo The name is $n
echo The value is $10val

val=10
echo The value is $val
```

Code Description

- `#!/bin/bash`
I have used this line as a shebang to specify that the script should be executed using the Bash interpreter.
- `echo "Hello World"`
I have printed a simple message to verify that the script runs successfully.
- `n=abcd`
I have declared a user-defined variable named `n` and assigned it the value `abcd`. In Bash, variables must not contain spaces around the `=` operator.
- `10val=10`
I attempted to declare a variable starting with a number. Bash does not allow variable names to begin with digits, so this results in an error.
- `echo The name is $n`
I have accessed the variable `n` using the `$` symbol, which performs variable expansion.

- `echo The value is $10val`
Since the variable name is invalid, Bash interprets `$10` as a positional parameter and appends `val` to it, producing unexpected behavior.
 - `val=10`
I correctly declared another variable named `val` and assigned it the value `10`.
 - `echo The value is $val`
I printed the correctly declared variable to verify proper assignment.
-

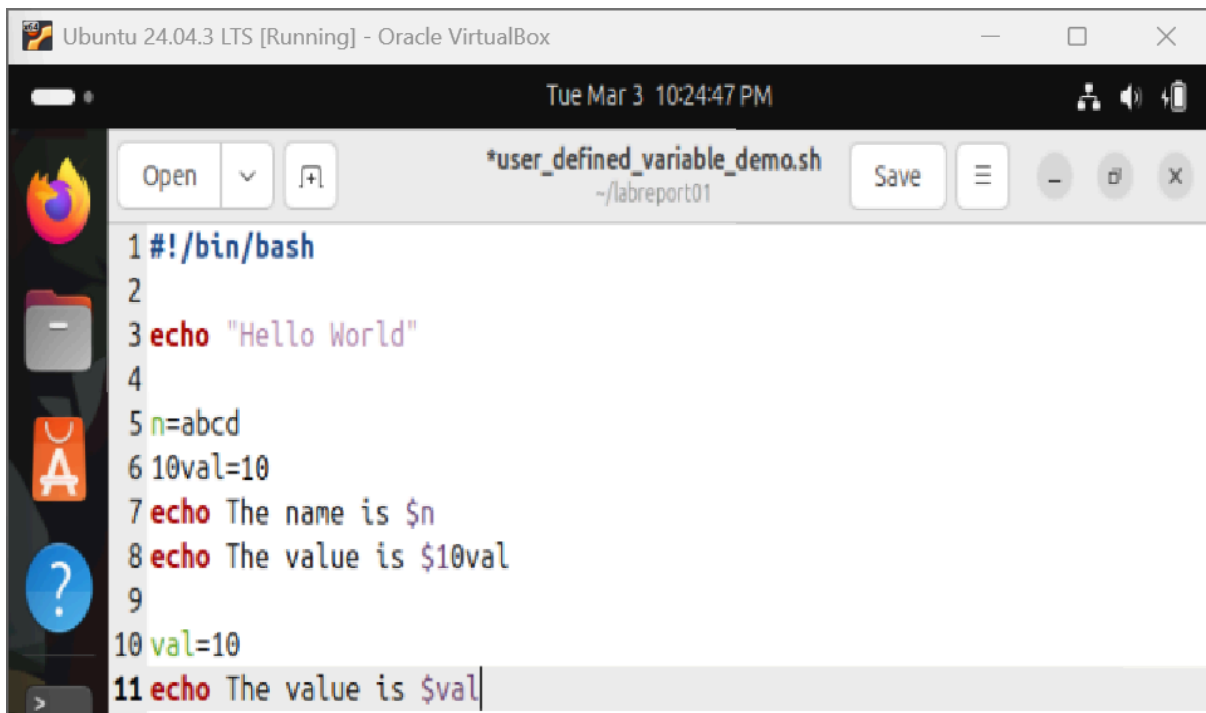
Execution

After saving the script file, I executed it using the following command in the terminal:

```
bash user_defined_variable_demo.sh
```

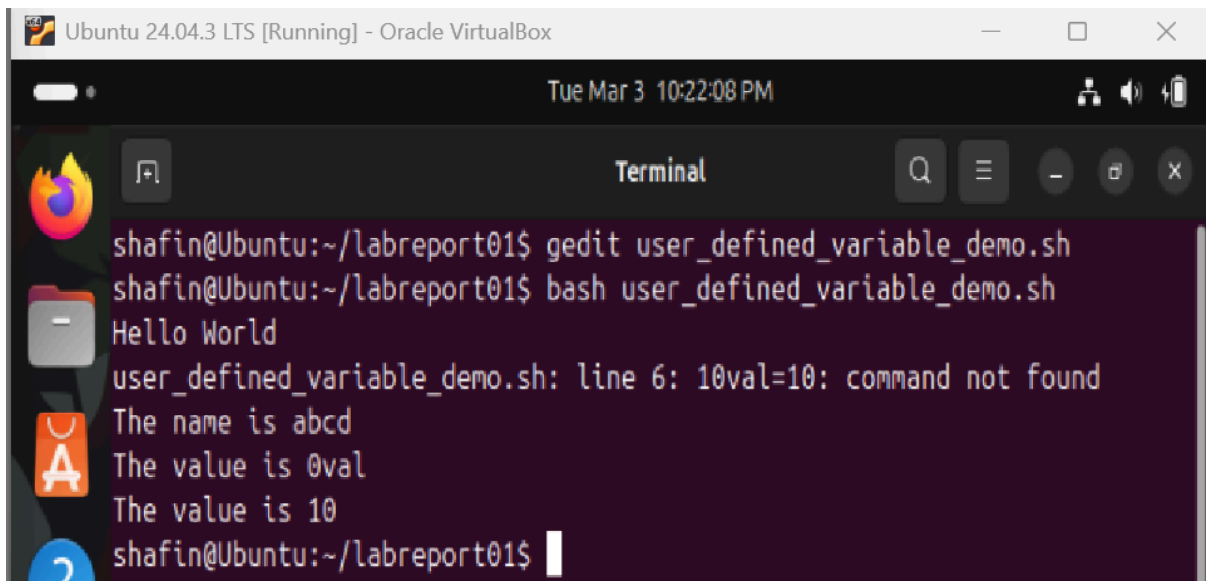
This command has instructed the Bash interpreter to read and execute the script file.

Output



```
1#!/bin/bash
2
3echo "Hello World"
4
5n=abcd
610val=10
7echo The name is $n
8echo The value is $10val
9
10val=10
11echo The value is $val
```

Figure 1: Text Editor Showing Source Code



```
shafin@Ubuntu:~/labreport01$ gedit user_defined_variable_demo.sh
shafin@Ubuntu:~/labreport01$ bash user_defined_variable_demo.sh
Hello World
user_defined_variable_demo.sh: line 6: 10val=10: command not found
The name is abcd
The value is oval
The value is 10
shafin@Ubuntu:~/labreport01$
```

Figure 2: Terminal Showing Script Execution

The output has displayed:

- The message “Hello World”
- The correct value of variable `n`
- An error message for invalid variable declaration (`10val=10`)
- The correct output for variable `val`

Discussion

Reflection

Through this experiment, I have learned how user-defined variables function in Bash scripting. I have understood that:

- Variable names cannot start with numbers.
- No spaces are allowed around the `=` operator.
- The `$` symbol is required to access variable values.
- Incorrect variable names produce syntax errors.

This experiment has helped me understand the internal working logic of variable parsing in the Bash shell.

Challenges Faced

During the implementation, I initially wrote variable assignments with spaces (e.g., `n = abcd`), which produced errors. I also observed that declaring a variable starting with a digit

caused a syntax error. I resolved these issues by correcting the syntax according to Bash variable naming rules.

Conclusion

In this experiment, I have successfully implemented and analyzed user-defined variables in a Bash shell script. I have understood the correct rules for variable declaration and the importance of proper syntax in scripting.

This knowledge has real-life applications in automation scripts, system administration tasks, DevOps operations, and software deployment processes, where variables are extensively used for configuration and dynamic data handling.

Experiment No: 02

Experiment Name: Implementation of Read Command in Bash Shell Script

Script Name:

read_command_input_demo.sh

Introduction

Overview

In this experiment, I have implemented a Bash shell script to understand the functionality of the `read` command. The primary objective of this experiment has been to learn how user input is taken dynamically from the terminal and stored in a user-defined variable.

I have demonstrated how interactive shell scripting works by prompting the user to enter a name and displaying the entered value.

Topics Covered

- `read` command in Bash
 - User input handling
 - Variable storage and expansion
 - Echo command usage
 - Interactive shell scripting
-

Environment / Platform

I have performed this experiment using the Ubuntu Linux operating system with the Bash shell environment.

To create and edit the script file, I have used the graphical text editor **gedit**. I opened the script file using the following command:

```
gedit read_command_input_demo.sh
```


After writing and saving the script, I executed it directly using the Bash interpreter through the terminal.

Task

Description

The task was to create a Bash script that prompts the user to enter a name. The script stores the entered value into a variable using the `read` command and then displays the stored value using the `echo` command.

Source Code

```
#!/bin/bash

echo "Enter name: "
read n
echo "Entered name: $n"
```

Code Description

- `#!/bin/bash`
I have used this line to specify that the script should be interpreted using the Bash shell.
 - `echo "Enter name: "`
I have displayed a message to prompt the user to enter a name.
 - `read n`
I have used the `read` command to accept input from the user. The entered value has been stored in the variable `n`.
 - `echo "Entered name: $n"`
I have displayed the value stored in variable `n` using variable expansion with the `$` symbol.
-

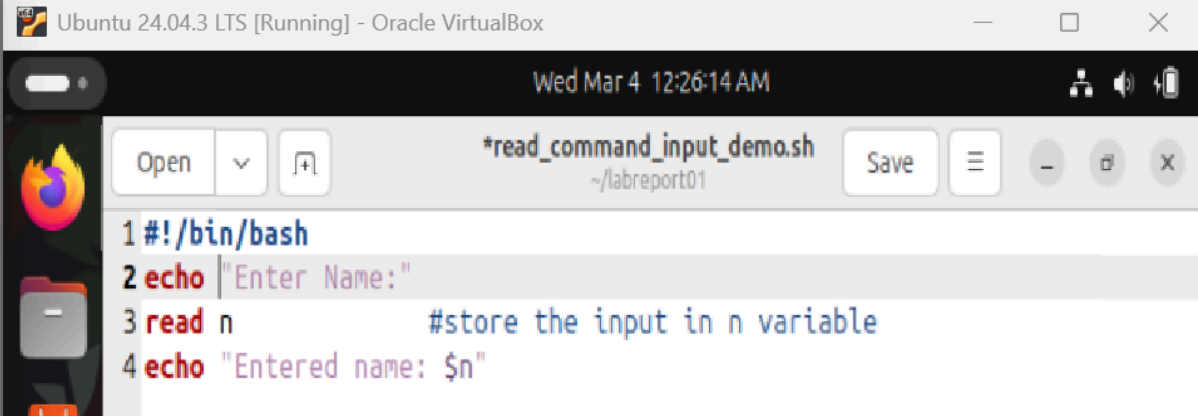
Execution

After saving the script file, I executed it using the following command:

bash read_command_input_demo.sh

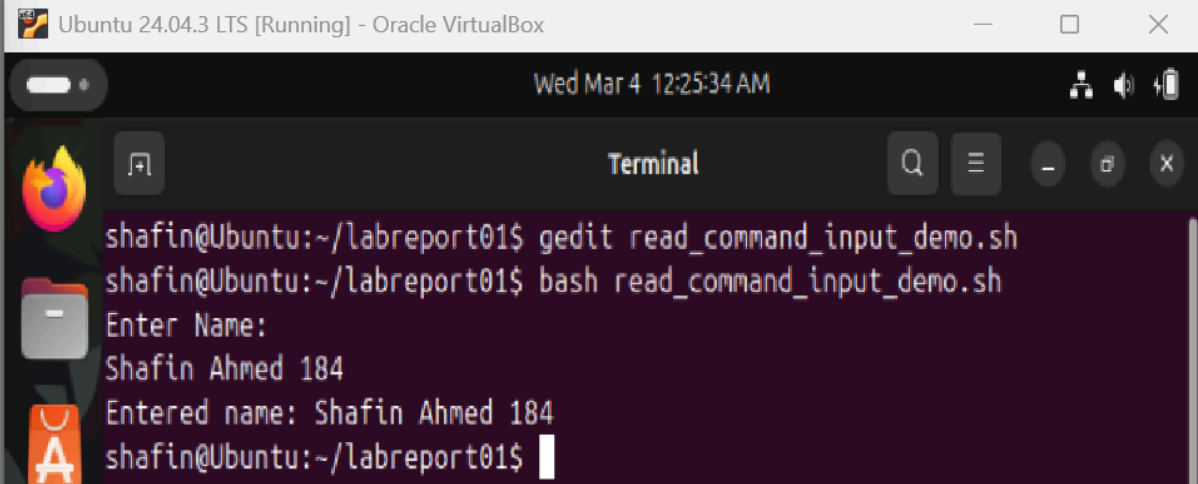
The script then prompted for input in the terminal.

Output



```
1 #!/bin/bash
2 echo "Enter Name:"
3 read n          #store the input in n variable
4 echo "Entered name: $n"
```

Figure 1: Text Editor (gedit) Showing Source Code



```
shafin@Ubuntu:~/labreport01$ gedit read_command_input_demo.sh
shafin@Ubuntu:~/labreport01$ bash read_command_input_demo.sh
Enter Name:
Shafin Ahmed 184
Entered name: Shafin Ahmed 184
shafin@Ubuntu:~/labreport01$
```

Figure 2: Terminal Showing User Input and Output

Sample Terminal Interaction:

```
Enter name:
abcd
Entered name: abcd
```

The script has successfully accepted user input and displayed the entered value correctly.

Discussion

Reflection

Through this experiment, I have learned how to create interactive shell scripts using the `read` command. I have understood how user input can be stored in variables and later accessed using variable expansion.

This experiment has helped me understand how dynamic data handling works in Bash scripting. I have also observed that the script waits for user input before proceeding to the next command.

Challenges Faced

Initially, I had to ensure that the variable name used in the `read` command matched the variable used in the `echo` statement. I carefully checked for spelling consistency to avoid logical errors. After verifying the syntax, the script executed successfully without any issues.

Conclusion

In this experiment, I have successfully implemented the `read` command in a Bash shell script. I have learned how to accept user input dynamically and store it in a variable for further processing.

This concept has real-life applications in system administration scripts, automation tools, login systems, and interactive command-line applications where user input is required.

Experiment No: 03

Experiment Name: Reading Input Using Default Variable in Bash

Script Name:

default_variable_read_demo.sh

Introduction

Overview

In this experiment, I have implemented a Bash shell script to understand how the `read` command works when no variable name is specified. The main objective of this experiment has been to learn about the default variable `$REPLY` in Bash.

I have demonstrated how user input is automatically stored in the built-in default variable when the `read` command is used without specifying a user-defined variable.

Topics Covered

- `read` command without variable name
 - Default variable `$REPLY`
 - User input handling
 - Variable expansion
 - Interactive shell scripting
-

Environment / Platform

I have performed this experiment using the Ubuntu Linux operating system in the Bash shell environment.

To create and edit the script file, I have used the graphical text editor **gedit**. I opened the file using the following command:

gedit default_variable_read_demo.sh

After writing and saving the script, I executed it directly from the terminal.

Since I executed the script using `./script_name`, I ensured that execution permission was granted using:

```
chmod +x default_variable_read_demo.sh
```

Task

Description

The task was to create a Bash script that accepts user input without specifying a variable name in the `read` command. The script then displays the entered input using the default variable `$REPLY`.

Source Code

```
#!/bin/bash

echo "Enter name: "
read
echo "Name : $REPLY"
```

Code Description

- `#!/bin/bash`
I have used this shebang line to specify that the script should be interpreted using the Bash shell.
 - `echo "Enter name: "`
I have displayed a prompt message asking the user to enter a name.
 - `read`
I have used the `read` command without specifying any variable name. In this case, Bash automatically stores the user input in the default variable `$REPLY`.
 - `echo "Name : $REPLY"`
I have accessed the stored input using `$REPLY` and displayed it in the terminal.
-

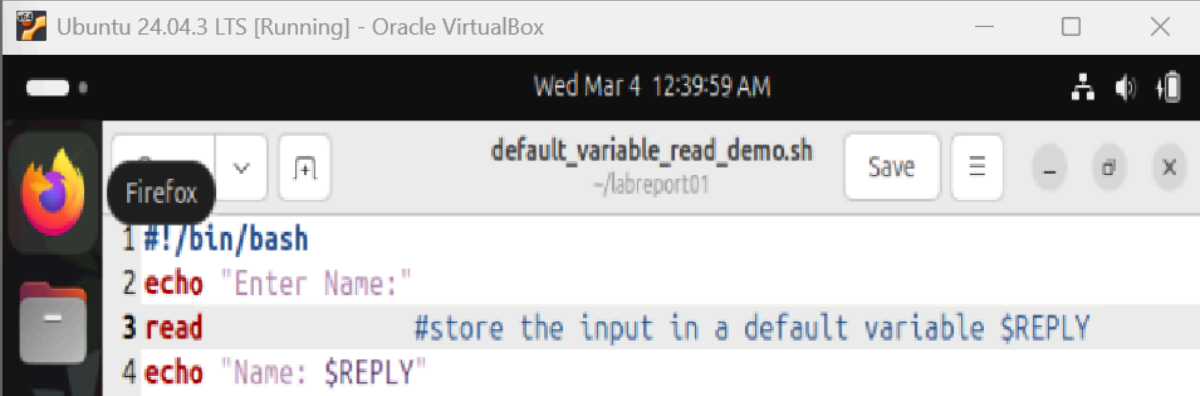
Execution

After saving the script file and granting execution permission, I executed it using the following command:

bash default_variable_read_demo.sh

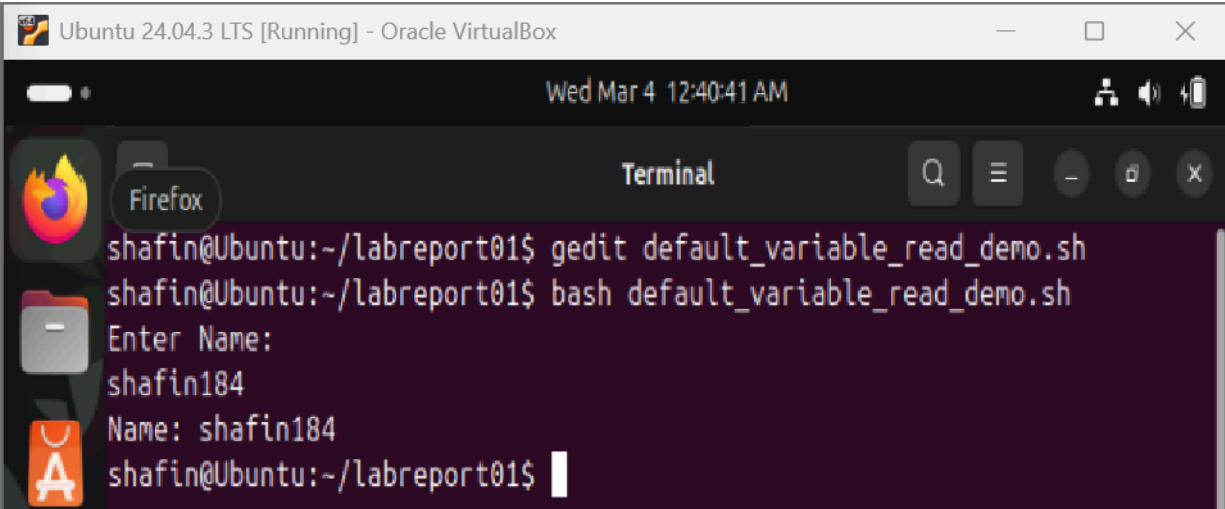
The script then prompts for user input in the terminal.

Output



```
1 #!/bin/bash
2 echo "Enter Name:"
3 read          #store the input in a default variable $REPLY
4 echo "Name: $REPLY"
```

Figure 1: Text Editor (gedit) Showing Source Code



```
shafin@Ubuntu:~/labreport01$ gedit default_variable_read_demo.sh
shafin@Ubuntu:~/labreport01$ bash default_variable_read_demo.sh
Enter Name:
shafin184
Name: shafin184
shafin@Ubuntu:~/labreport01$
```

Figure 2: Terminal Showing Execution Using ./ Command

Sample Terminal Interaction:

```
Enter name:
shafin184
Name : shafin184
```

The script has successfully accepted user input and displayed the value stored in the default variable `$REPLY`.

Discussion

Reflection

Through this experiment, I have learned that when no variable name is provided with the `read` command, Bash automatically stores the input in a special built-in variable called `$REPLY`. This has helped me understand internal variable handling in Bash scripting.

I have observed that `$REPLY` behaves like a normal variable and can be accessed using the `$` symbol for variable expansion.

Challenges Faced

Initially, I was unsure where the input would be stored when no variable name was specified. After studying the Bash documentation and testing the script, I understood that `$REPLY` is the default storage variable. Once I accessed `$REPLY`, the script worked correctly without any issues.

Conclusion

In this experiment, I have successfully implemented input handling using the default variable `$REPLY` in Bash scripting. I have learned how Bash internally manages user input when no explicit variable is defined.

This concept is useful in automation scripts and interactive shell programs where quick input handling is required without declaring additional variables.

Experiment No: 04

Experiment Name: Reading Multiple Inputs Using Read Command in Bash

Script Name: multiple_input_read_demo.sh

Introduction

Overview

In this experiment, I have implemented a Bash shell script to understand how multiple inputs can be read from the user using a single `read` command. The primary objective of this experiment has been to learn how Bash stores multiple values into separate variables when entered in a single line.

I have demonstrated how user input separated by spaces can be distributed across multiple variables dynamically.

Topics Covered

- `read` command with multiple variables
 - Handling multiple user inputs
 - Variable assignment from a single input line
 - Variable expansion in Bash
 - Interactive shell scripting
-

Environment / Platform

I have performed this experiment using the Ubuntu Linux operating system in the Bash shell environment.

To create and edit the script file, I have used the graphical text editor **gedit**. I opened the script file using the following command:

```
gedit multiple_input_read_demo.sh
```


After writing and saving the script, I executed it directly from the terminal using the Bash interpreter.

Task

Description

The task was to create a Bash script that accepts three names from the user in a single line. The script stores each input into separate variables using the `read` command and then displays the stored values using the `echo` command.

Source Code

```
#!/bin/bash

echo "Enter names: "
read n1 n2 n3
echo "Names: $n1 , $n2 , $n3"
```

Code Description

- `#!/bin/bash`
I have used this shebang line to specify that the script should be executed using the Bash interpreter.
 - `echo "Enter names: "`
I have displayed a message prompting the user to enter three names separated by spaces.
 - `read n1 n2 n3`
I have used the `read` command with three variables. Bash automatically splits the input based on spaces and assigns the first value to `n1`, the second value to `n2`, and the third value to `n3`.
 - `echo "Names: $n1 , $n2 , $n3"`
I have displayed the values stored in each variable using variable expansion.
-

Execution

After saving the script file, I executed it using the following command:

bash multiple_input_read_demo.sh

The script then prompted the user to enter multiple inputs in one line.

Output

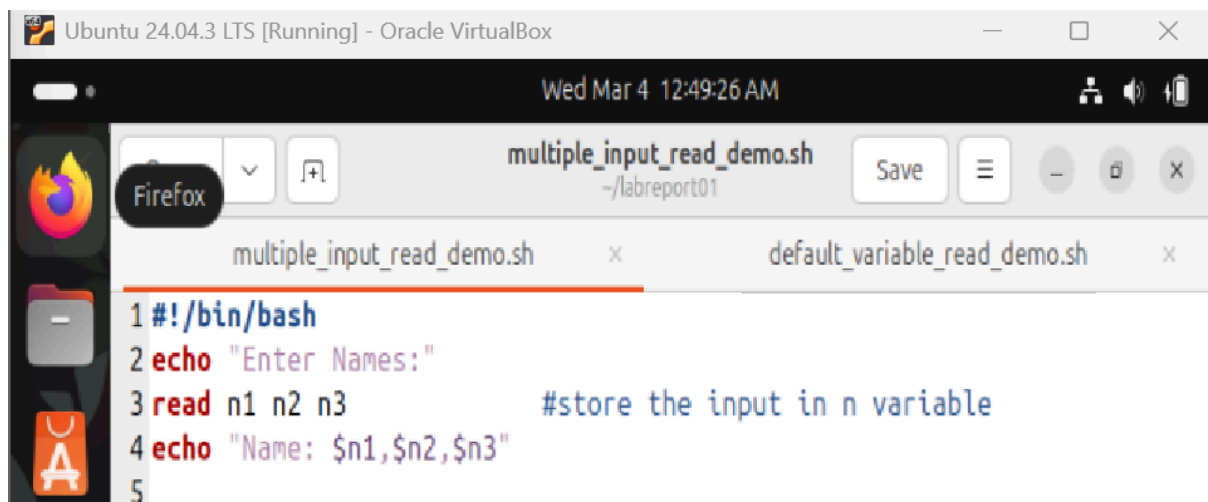


Figure 1: Text Editor (gedit) Showing Source Code

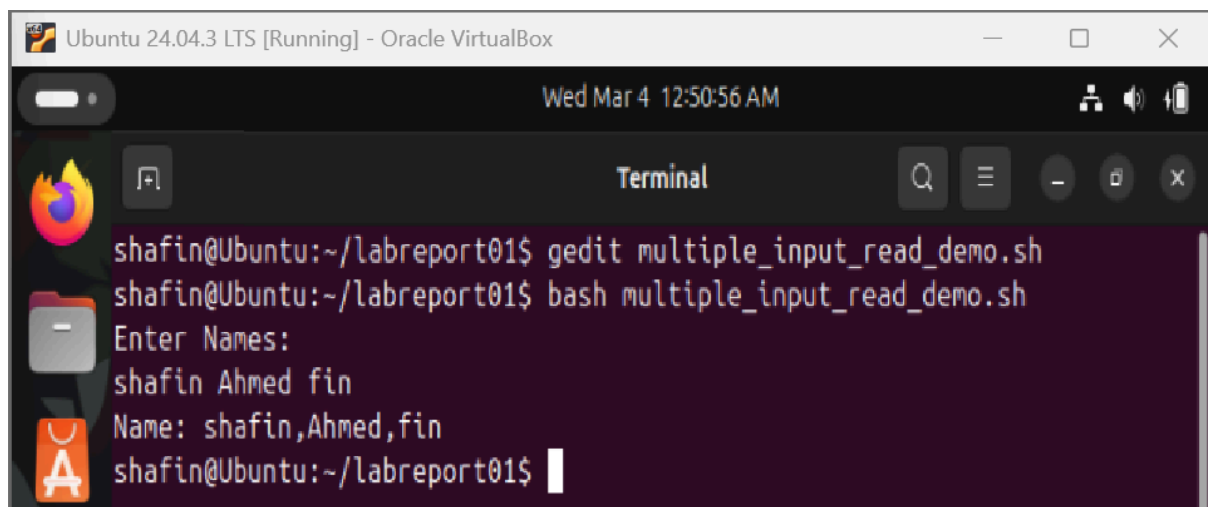


Figure 2: Terminal Showing Multiple Input and Output

The script has successfully stored and displayed multiple inputs entered in a single line.

Discussion

Reflection

Through this experiment, I have learned how the `read` command can accept multiple inputs and assign them to different variables simultaneously. I have understood that Bash separates input values using spaces by default and distributes them sequentially across the listed variables.

This experiment has strengthened my understanding of dynamic input handling in shell scripting.

Challenges Faced

Initially, I needed to ensure that the number of variables matched the number of expected inputs. If fewer inputs were provided, some variables remained empty. I resolved this by carefully entering three values separated by spaces during execution.

Conclusion

In this experiment, I have successfully implemented a Bash script that reads multiple user inputs using a single `read` command. I have learned how Bash automatically assigns space-separated values to corresponding variables.

This concept has practical applications in form handling, automation scripts, configuration setups, and system administration tasks where multiple inputs are required from the user at once.

Experiment No: 05

Experiment Name: Reading Multiple Inputs Using Array in Bash

Script Name: array_input_read_demo.sh

Introduction

Overview

In this experiment, I have implemented a Bash shell script to understand how multiple inputs can be stored in an array using the `read` command. The primary objective of this experiment has been to learn how arrays function in Bash scripting and how multiple user inputs can be dynamically stored and accessed using index positions.

I have demonstrated how the `-a` option of the `read` command stores space-separated input values into an array.

Topics Covered

- Arrays in Bash
 - `read -a` option
 - Storing input in arrays
 - Accessing array elements using index
 - Variable expansion with array notation
-

Environment / Platform

I have performed this experiment using the Ubuntu Linux operating system in the Bash shell environment.

To create and edit the script file, I have used the graphical text editor **gedit**. I opened the script file using the following command:

```
gedit array_input_read_demo.sh
```

After writing and saving the script, I executed it from the terminal using the Bash interpreter.

Task

Description

The task was to create a Bash script that accepts multiple names from the user in a single line and stores them in an array using the `read -a` command. The script then displays individual array elements using index-based access.

Source Code

```
#!/bin/bash

echo "Enter names: "
read -a n
echo "Names: ${n[0]}, ${n[1]}, ${n[2]}"
```

Code Description

- `#!/bin/bash`
I have used this shebang line to specify that the script should be interpreted using the Bash shell.
 - `echo "Enter names: "`
I have displayed a prompt asking the user to enter multiple names separated by spaces.
 - `read -a n`
I have used the `-a` option with the `read` command to store the input values into an array named `n`. Bash automatically splits the input based on spaces and stores each value in sequential index positions starting from 0.
 - `echo "Names: ${n[0]}, ${n[1]}, ${n[2]}"`
I have accessed the first three elements of the array using index notation and displayed them using variable expansion.
-

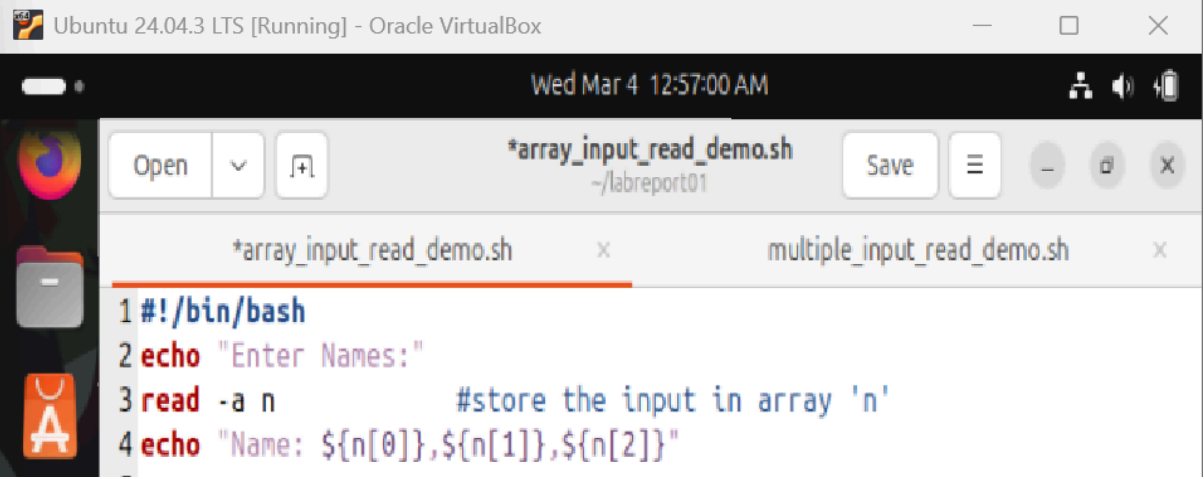
Execution

After saving the script file, I executed it using the following command:

```
bash array_input_read_demo.sh
```

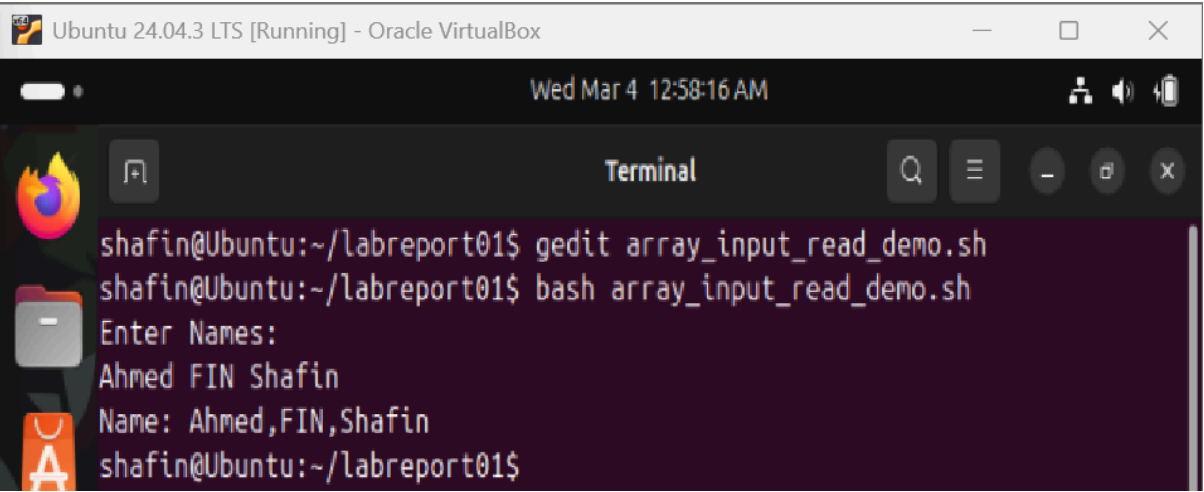
The script then prompted the user to enter multiple values.

Output



```
1 #!/bin/bash
2 echo "Enter Names:"
3 read -a n          #store the input in array 'n'
4 echo "Name: ${n[0]},${n[1]},${n[2]}"
```

Figure 1: Text Editor (gedit) Showing Source Code



```
shafin@Ubuntu:~/labreport01$ gedit array_input_read_demo.sh
shafin@Ubuntu:~/labreport01$ bash array_input_read_demo.sh
Enter Names:
Ahmed FIN Shafin
Name: Ahmed,FIN,Shafin
shafin@Ubuntu:~/labreport01$
```

Figure 2: Terminal Showing Array Input and Output

The script has successfully stored the input values inside an array and displayed them using index-based access.

Discussion

Reflection

Through this experiment, I have learned how arrays are implemented in Bash scripting. I have understood that the `read -a` option allows multiple inputs to be stored in an indexed array structure. I have also learned how array elements are accessed using `${array_name[index]}` syntax.

This experiment has enhanced my understanding of structured data storage in shell scripting.

Challenges Faced

Initially, I needed to understand how array indexing works in Bash, especially that indexing starts from 0. I carefully tested the script by entering multiple inputs to verify correct storage and retrieval of array elements. After verifying the indexing method, the script executed successfully without issues.

Conclusion

In this experiment, I have successfully implemented a Bash script that reads multiple inputs and stores them in an array. I have learned how to use the `read -a` command and access array elements using index positions.

This concept has practical applications in automation scripts, configuration processing, bulk data handling, and system administration tasks where multiple values must be managed efficiently.

Experiment No: 06

Experiment Name: Implementation of If-Else Conditional Statement in Bash

Script Name:

conditional_if_statement_demo.sh

Introduction

Overview

In this experiment, I have implemented a Bash shell script to understand the working mechanism of conditional statements using the `if`, `elif`, and `else` structure. The primary objective of this experiment has been to learn how decision-making is performed in shell scripting based on user input.

I have demonstrated how different conditions are evaluated and how the script executes different blocks of code depending on the result of the comparison.

Topics Covered

- `if` statement in Bash
 - `elif` and `else` conditions
 - String comparison using `[ ]`
 - User input handling with `read`
 - Conditional execution flow
-

Environment / Platform

I have performed this experiment using the Ubuntu Linux operating system in the Bash shell environment.

To create and edit the script file, I have used the graphical text editor **gedit**. I opened the script file using the following command:

gedit conditional_if_statement_demo.sh

After writing and saving the script, I executed it using the Bash interpreter in the terminal.

Task

Description

The task was to create a Bash script that accepts a word from the user and evaluates it using conditional statements. Based on the entered value, the script displays a specific message.

If the user enters "y", a message indicating that condition y is true is displayed.

If the user enters "x", a message indicating that condition x is true is displayed.

For any other input, a default message is shown.

Source Code

```
#!/bin/bash

read word
echo "Enter: $word"

if [ $word == "y" ]
then
    echo "condition y is true"
elif [ $word == "x" ]
then
    echo "condition x is true"
else
    echo "condition is false"
fi
```

Code Description

- `#!/bin/bash`
I have used this shebang line to specify that the script should be executed using the Bash interpreter.
- `read word`
I have accepted input from the user and stored it in the variable `word`.

- `echo "Enter: $word"`
I have displayed the entered word to confirm the received input.
- `if [ $word == "y" ]`
I have used an `if` condition to check whether the entered word is equal to "y".
- `elif [ $word == "x" ]`
I have used the `elif` statement to check an alternative condition if the first condition evaluates to false.
- `else`
I have provided a default execution block for all other inputs.
- `fi`
I have used `fi` to terminate the `if` conditional block.

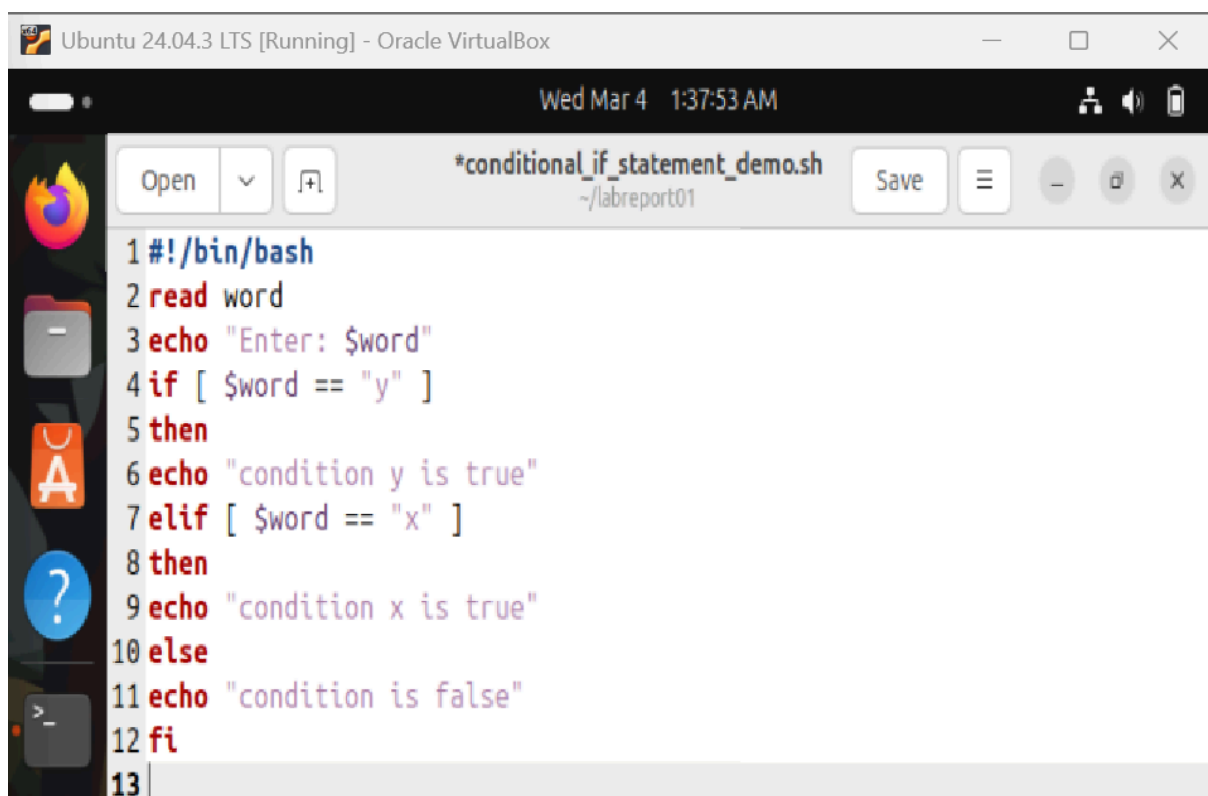
Execution

After saving the script file, I executed it using the following command:

`bash conditional_if_statement_demo.sh`

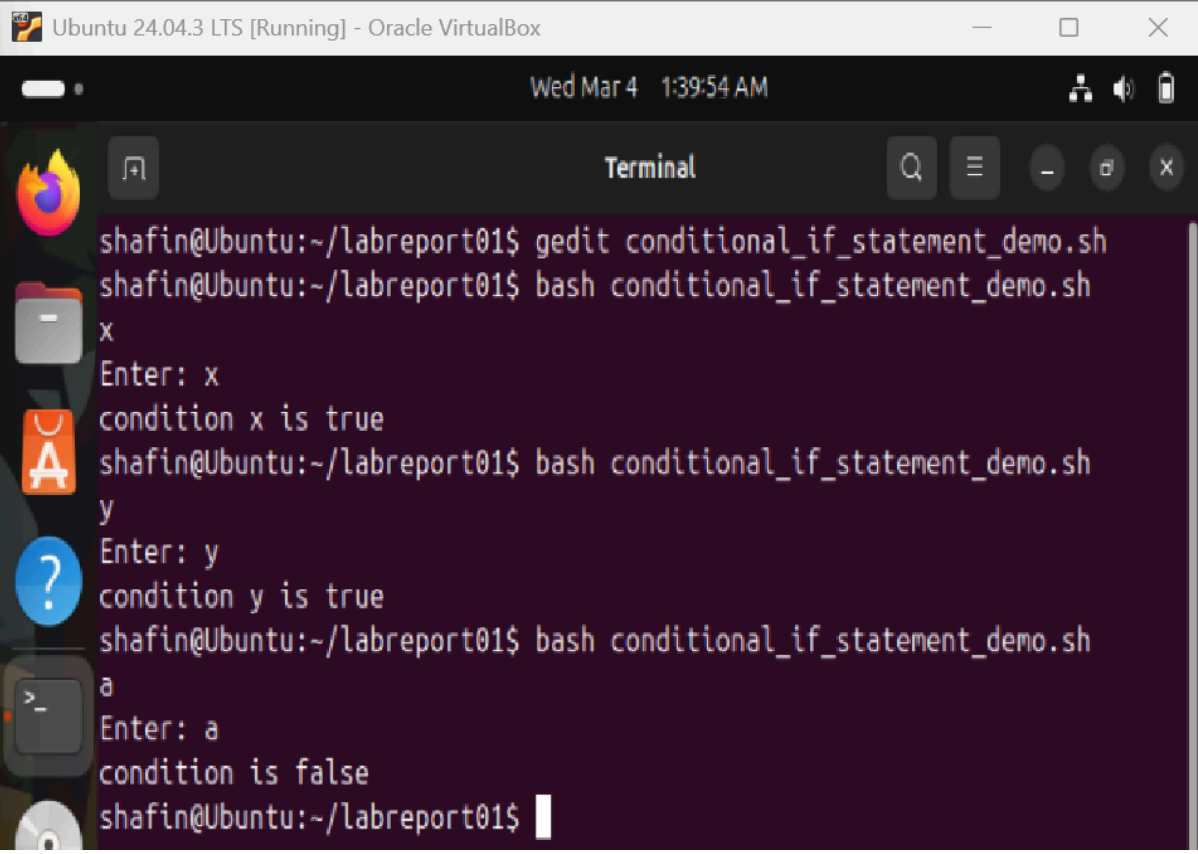
The script then waited for user input and evaluated the condition accordingly.

Output



```
1 #!/bin/bash
2 read word
3 echo "Enter: $word"
4 if [ $word == "y" ]
5 then
6 echo "condition y is true"
7 elif [ $word == "x" ]
8 then
9 echo "condition x is true"
10 else
11 echo "condition is false"
12 fi
13 |
```

Figure 1: Text Editor (gedit) Showing Source Code

A screenshot of a terminal window titled "Ubuntu 24.04.3 LTS [Running] - Oracle VirtualBox". The window shows a user named "shafin" at a prompt in the directory "~/labreport01". The user runs "gedit conditional_if_statement_demo.sh" and then "bash conditional_if_statement_demo.sh". The script prompts for input. When the user enters "x", the script outputs "condition x is true". When the user enters "y", the script outputs "condition y is true". When the user enters "a", the script outputs "condition is false". The terminal window has a dark background and a sidebar with application icons on the left.

```
shafin@Ubuntu:~/labreport01$ gedit conditional_if_statement_demo.sh
shafin@Ubuntu:~/labreport01$ bash conditional_if_statement_demo.sh
x
Enter: x
condition x is true
shafin@Ubuntu:~/labreport01$ bash conditional_if_statement_demo.sh
y
Enter: y
condition y is true
shafin@Ubuntu:~/labreport01$ bash conditional_if_statement_demo.sh
a
Enter: a
condition is false
shafin@Ubuntu:~/labreport01$
```

Figure 2: Terminal Showing Conditional Execution

The script has successfully evaluated different conditions based on the user input.

Discussion

Reflection

Through this experiment, I have learned how decision-making is implemented in Bash scripting using conditional statements. I have understood how string comparison is performed inside square brackets `[ ]` and how execution flow changes depending on logical evaluation.

This experiment has helped me understand the importance of control structures in programming and how scripts can respond dynamically to different inputs.

Challenges Faced

Initially, I needed to ensure proper spacing inside the conditional brackets `[ ]`, as Bash requires spaces around operators for correct syntax. After verifying the syntax carefully, the script executed successfully without errors.

Conclusion

In this experiment, I have successfully implemented an `if-elif-else` conditional statement in a Bash shell script. I have learned how logical conditions are evaluated and how different execution blocks are selected based on user input.

This concept has practical applications in automation scripts, validation systems, menu-driven programs, and system configuration tools where decision-making is essential.

Experiment No: 07

Experiment Name: Implementation of While Loop in Bash

Script Name: while.sh

Introduction

Overview

In this experiment, I have implemented a Bash shell script to understand the working mechanism of the `while` loop. The primary objective of this experiment has been to learn how iterative execution is performed in shell scripting using looping constructs.

I have demonstrated how a loop repeatedly executes a block of code as long as a specified condition remains true.

Topics Covered

- `while` loop in Bash
- Numeric comparison using `-le`
- Loop execution flow
- `sleep` command usage
- Manual termination using `Ctrl + C`

Environment / Platform

I have performed this experiment using the Ubuntu Linux operating system in the Bash shell environment.

To create and edit the script file, I have used the graphical text editor **gedit**. I opened the file using the following command:

gedit while.sh

After writing and saving the script, I executed it using the Bash interpreter from the terminal.

Task

Description

The task was to create a Bash script that initializes a variable and repeatedly prints its value using a `while` loop. The loop continues executing as long as the condition remains true. Since the loop does not modify the variable inside the loop body, it runs continuously and must be terminated manually.

Source Code

```
#!/bin/bash
n=1
while [ $n -le 5 ]
do
    echo "$n"
    sleep 3
done
```

Code Description

- `#!/bin/bash`
I have used this shebang line to specify that the script should be interpreted using the Bash shell.
- `n=1`
I have initialized a variable `n` with the value 1.
- `while [ $n -le 5 ]`
I have used a `while` loop to check whether the value of `n` is less than or equal to 5. The `-le` operator performs numeric comparison.
- `echo "$n"`
I have printed the value of `n` inside the loop.
- `sleep 3`
I have used the `sleep` command to pause execution for 3 seconds between each iteration.
- `done`
I have used `done` to terminate the `while` loop block.

Since the value of `n` has not been incremented inside the loop, the condition has always remained true. Therefore, the loop continued executing repeatedly.

Execution

After saving the script file, I executed it using the following command:

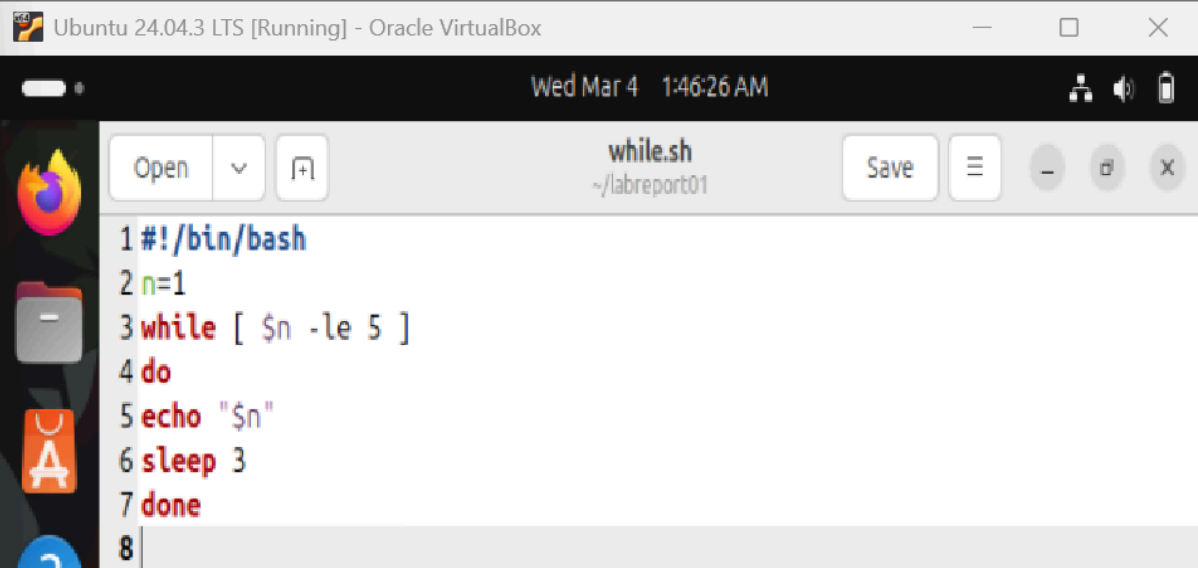
bash while.sh

The loop repeatedly printed the value **1** every 3 seconds.

To stop the infinite loop, I terminated the execution manually using:

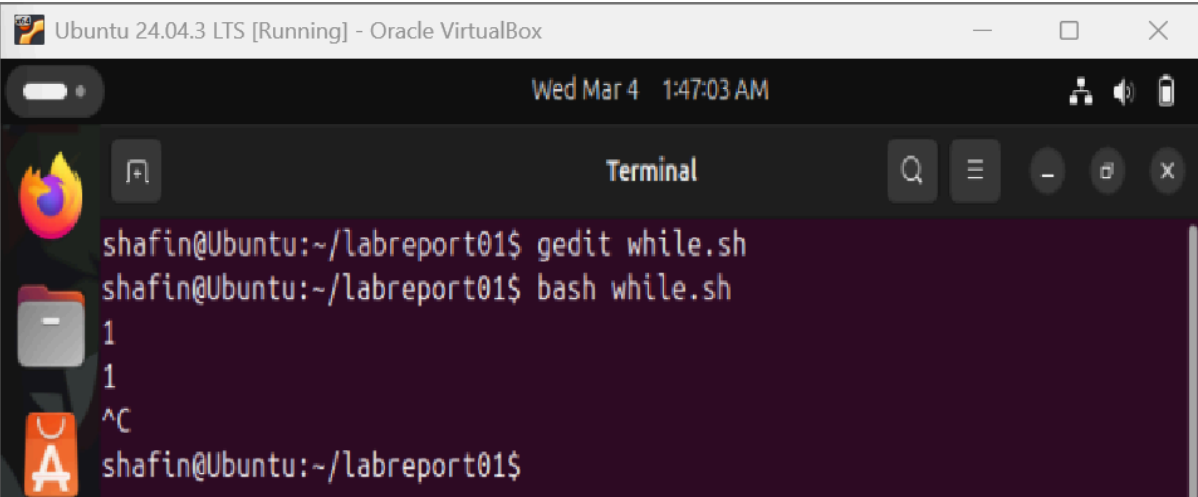
Ctrl + C

Output



```
1#!/bin/bash
2n=1
3while [ $n -le 5 ]
4do
5echo "$n"
6sleep 3
7done
8
```

Figure 1: Text Editor (gedit) Showing Source Code



```
shafin@Ubuntu:~/labreport01$ gedit while.sh
shafin@Ubuntu:~/labreport01$ bash while.sh
1
1
^C
shafin@Ubuntu:~/labreport01$
```

Figure 2: Terminal Showing Continuous Loop Execution

The output has continuously displayed the value **1** at intervals of 3 seconds until the execution was manually stopped.

Discussion

Reflection

Through this experiment, I have learned how the `while` loop works in Bash scripting. I have understood that a loop continues executing as long as the specified condition evaluates to true. I have also observed that if the loop variable is not modified inside the loop, the condition may remain permanently true, resulting in an infinite loop.

This experiment has helped me understand the importance of properly updating loop variables.

Challenges Faced

Initially, I expected the loop to stop automatically after reaching 5. However, since the variable `n` was not incremented inside the loop, the condition never changed. I identified this logical issue and understood that loop control variables must be updated to avoid infinite execution.

Conclusion

In this experiment, I have successfully implemented a `while` loop in a Bash shell script. I have learned how iterative execution works and how conditions control loop behavior. I have also understood how infinite loops can occur if loop variables are not updated properly.

This concept has practical applications in automation processes, monitoring scripts, background services, and system administration tasks where repeated execution is required.

Experiment No: 08

Experiment Name: Implementation of For Loop in Bash

Script Name: for.sh

Introduction

Overview

In this experiment, I have implemented a Bash shell script to understand the working mechanism of the `for` loop. The primary objective of this experiment has been to learn how iteration over a sequence of numbers can be performed using a numeric `for` loop in Bash scripting.

I have demonstrated how the loop executes a block of code repeatedly for a defined number of times.

Topics Covered

- `for` loop in Bash
 - C-style numeric iteration
 - Loop initialization, condition, and increment
 - Iterative execution of commands
 - `echo` command usage
-

Environment / Platform

I have performed this experiment using the Ubuntu Linux operating system in the Bash shell environment.

To create and edit the script file, I have used the graphical text editor **gedit**. I opened the script file using the following command:

gedit for.sh

After writing and saving the script, I executed it using the Bash interpreter from the terminal.

Task

Description

The task was to create a Bash script that prints numbers from 0 to 4 using a `for` loop. The loop iterates a specific number of times defined by the initialization, condition, and increment steps in the C-style `for` loop syntax.

Source Code

```
#!/bin/bash
for (( i=0; i<5; i++ ))
do
    echo $i;
done
```

Code Description

- `#!/bin/bash`
I have used this shebang line to specify that the script should be executed using the Bash interpreter.
 - `for (( i=0; i<5; i++ ))`
I have used a C-style `for` loop where `i` is initialized to 0, the loop continues while `i` is less than 5, and `i` is incremented by 1 after each iteration.
 - `echo $i`
I have printed the current value of `i` during each iteration of the loop.
 - `done`
I have used `done` to terminate the `for` loop block.
-

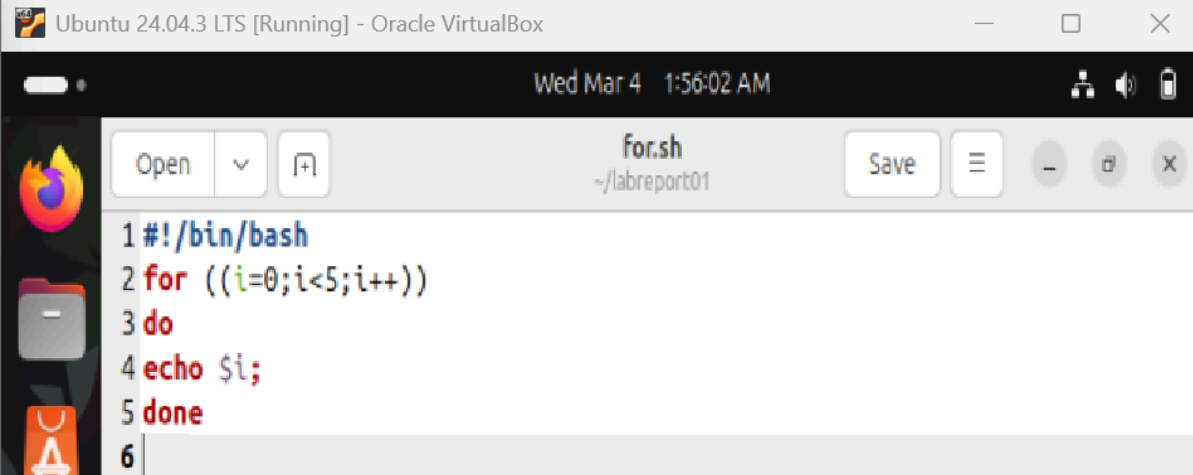
Execution

After saving the script file, I executed it using the following command:

bash for.sh

The script then printed the numbers from 0 to 4 sequentially in the terminal.

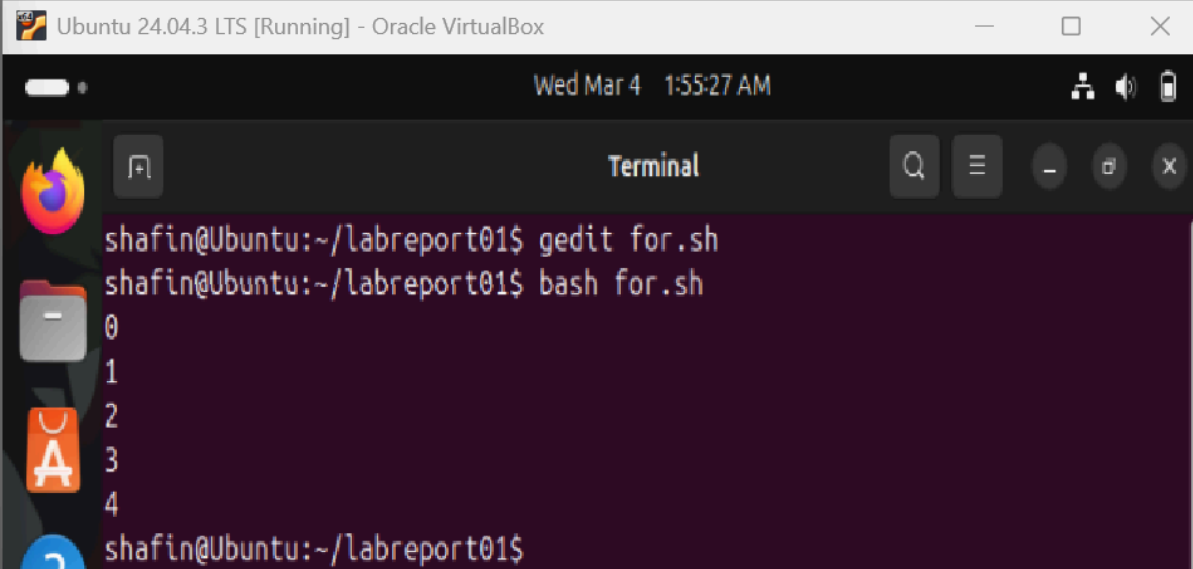
Output



The screenshot shows a window titled 'Ubuntu 24.04.3 LTS [Running] - Oracle VirtualBox'. Inside, a text editor window titled 'for.sh' is open, showing the following code:

```
1 #!/bin/bash
2 for ((i=0;i<5;i++))
3 do
4 echo $i;
5 done
6
```

Figure 1: Text Editor (gedit) Showing Source Code



The screenshot shows a terminal window titled 'Terminal' with the following commands and output:

```
shafin@Ubuntu:~/labreport01$ gedit for.sh
shafin@Ubuntu:~/labreport01$ bash for.sh
0
1
2
3
4
shafin@Ubuntu:~/labreport01$
```

Figure 2: Terminal Showing Loop Execution

The output has successfully displayed all numbers from 0 to 4 as defined by the loop conditions.

Discussion

Reflection

Through this experiment, I have learned how to implement a numeric `for` loop in Bash scripting. I have understood the importance of loop initialization, termination conditions, and increment/decrement steps in controlling loop execution.

This experiment has strengthened my understanding of iterative control structures in shell scripting.

Challenges Faced

Initially, I had to carefully ensure the syntax of the C-style `for` loop, including the correct use of double parentheses `(( ))` and proper spacing. After verifying the syntax, the script executed successfully without errors.

Conclusion

In this experiment, I have successfully implemented a `for` loop in a Bash shell script to print a sequence of numbers. I have learned how loops provide a mechanism to execute repetitive tasks efficiently.

This concept has practical applications in automation scripts, batch processing, repetitive calculations, and system administration tasks where repetitive execution is required.

Experiment No: 09

Experiment Name: Implementation of Until Loop in Bash

Script Name: until.sh

Introduction

Overview

In this experiment, I have implemented a Bash shell script to understand the working mechanism of the `until` loop. The primary objective of this experiment has been to learn how iterative execution is performed in Bash until a specified condition becomes true.

I have demonstrated how the `until` loop repeatedly executes a block of code as long as the condition evaluates to false.

Topics Covered

- `until` loop in Bash
 - Numeric comparison using `-ge`
 - Loop execution flow
 - Incrementing variables inside a loop
 - `echo` command usage
-

Environment / Platform

I have performed this experiment using the Ubuntu Linux operating system in the Bash shell environment.

To create and edit the script file, I have used the graphical text editor **gedit**. I opened the script file using the following command:

gedit until.sh

After writing and saving the script, I executed it from the terminal using the Bash interpreter.

Task

Description

The task was to create a Bash script that initializes a variable and repeatedly prints its value using an `until` loop. The loop continues executing as long as the condition evaluates to false. Once the condition becomes true, the loop terminates.

I have incremented the variable inside the loop to eventually satisfy the termination condition.

Source Code

```
#!/bin/bash
n=0
until [ $n -ge 10 ]
do
    echo $n
    n=$((n+1))
done
```

Code Description

- `#!/bin/bash`
I have used this shebang line to specify that the script should be executed using the Bash interpreter.
- `n=0`
I have initialized a variable `n` with the value 0.
- `until [ $n -ge 10 ]`
I have used an `until` loop that executes the code block as long as the condition `$n -ge 10` evaluates to false.
- `echo $n`
I have printed the current value of `n` in each iteration.
- `n=$((n+1))`
I have incremented the variable `n` by 1 in each iteration to ensure the loop eventually terminates.
- `done`
I have used `done` to terminate the `until` loop block.

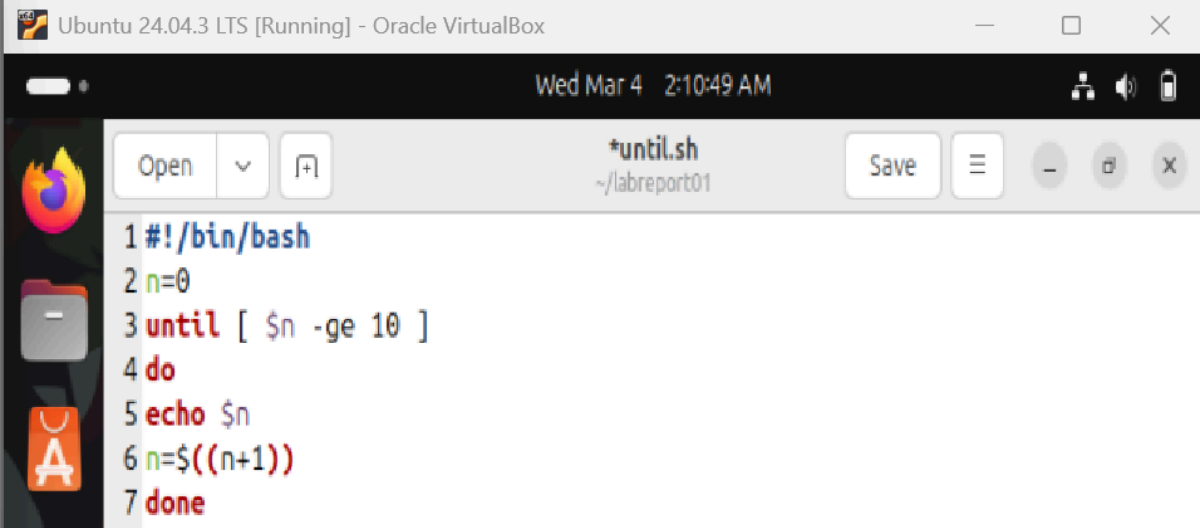
Execution

After saving the script file, I executed it using the following command:

bash until.sh

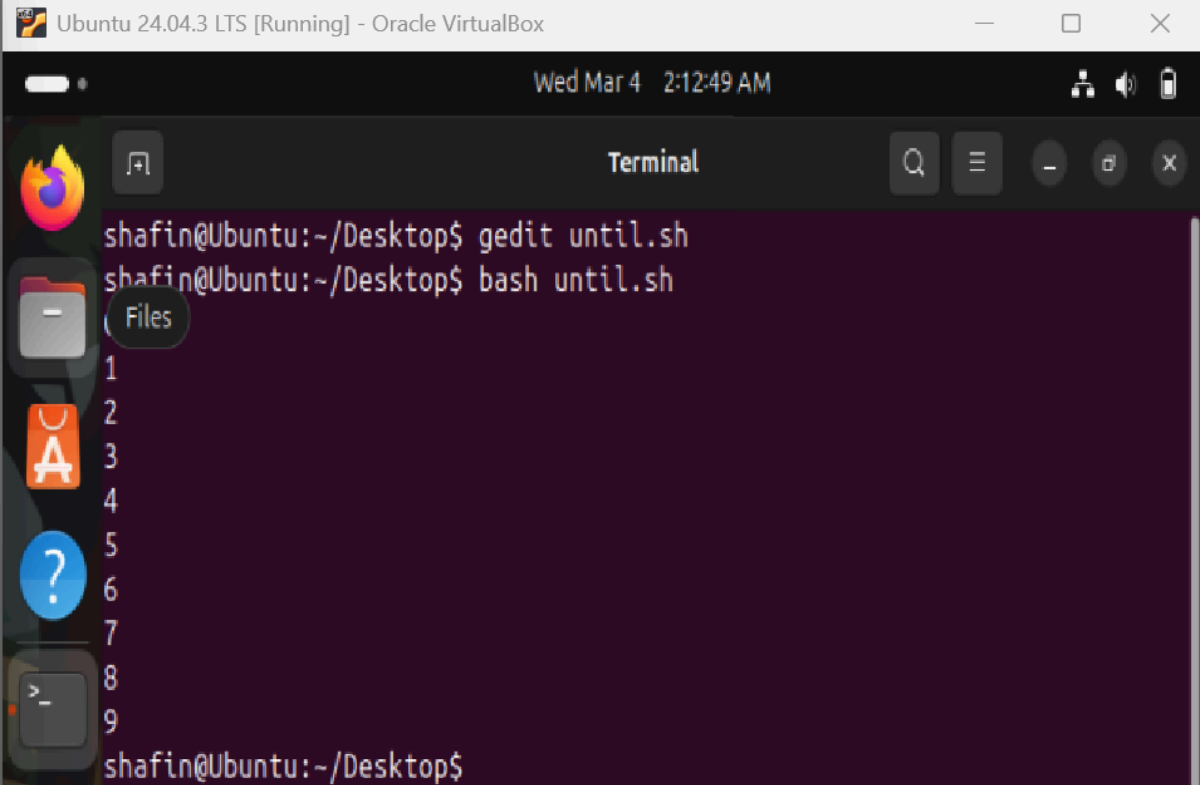
The script then printed numbers from 0 to 9 sequentially in the terminal.

Output



```
1 #!/bin/bash
2 n=0
3 until [ $n -ge 10 ]
4 do
5 echo $n
6 n=$((n+1))
7 done
```

Figure 1: Text Editor (gedit) Showing Source Code



```
shafin@Ubuntu:~/Desktop$ gedit until.sh
shafin@Ubuntu:~/Desktop$ bash until.sh
1
2
3
4
5
6
7
8
9
shafin@Ubuntu:~/Desktop$
```

Figure 2: Terminal Showing Loop Execution

The output has successfully displayed all numbers from 0 to 9 as defined by the loop conditions.

Discussion

Reflection

Through this experiment, I have learned how the `until` loop works in Bash scripting. I have understood that an `until` loop executes repeatedly as long as the condition evaluates to false, which is the opposite behavior of a `while` loop. I have also learned how to increment loop variables inside the loop to avoid infinite execution.

This experiment has strengthened my understanding of iterative control structures in shell scripting.

Challenges Faced

Initially, I wrote `n = $( ( n+1 ) )` with spaces, which caused a syntax error. I corrected it to `n=$((n+1))` without spaces. After correcting the syntax, the script executed successfully.

Conclusion

In this experiment, I have successfully implemented an `until` loop in a Bash shell script. I have learned how loop execution is controlled based on a false condition and how proper variable management ensures correct termination.

This concept has practical applications in automation scripts, repeated monitoring tasks, and iterative data processing in system administration and shell programming.

Experiment No: 10

Experiment Name: Arithmetic Operations in Bash

Script Names:

- `arithmetic_demo1.sh`
- `arithmetic_demo2.sh`
- `arithmetic_demo3.sh`
- `arithmetic_demo4.sh`

Introduction

Overview

In this experiment, I have implemented Bash shell scripts to perform different arithmetic operations. The primary objective of this experiment has been to learn how Bash handles numeric computations, both with integers and floating-point numbers, using built-in arithmetic expansion, `expr`, and the `bc` command.

I have demonstrated multiple methods of performing arithmetic operations, including addition, subtraction, multiplication, division, modulo, and square root operations.

Topics Covered

- Integer arithmetic using `$(( ))`
- Integer arithmetic using `expr`
- Floating-point arithmetic using `bc`
- Using the math library with `bc -l`
- Scale and precision in floating-point division
- Arithmetic operation commands in shell scripting

Environment / Platform

I have performed this experiment using Ubuntu Linux in the Bash shell environment.

I have used the **gedit** text editor to write scripts and executed them in the terminal using Bash.

Task

Description

The task was divided into four scripts:

1. **Basic integer arithmetic using `$(( ))`**
2. **Integer arithmetic using `expr`**
3. **Floating-point arithmetic using `bc`**
4. **Advanced floating-point arithmetic using `bc -l`**

I have calculated addition, subtraction, multiplication, division, modulo, and square root operations using different methods.

Source Codes

Script 1: arithmetic_demo1.sh (Integer Arithmetic using `$(( ))`)

```
#!/bin/bash
echo 1+1

num1=30
num2=6
echo $(( num1 + num2 ))
echo $(( num1 - num2 ))
echo $(( num1 * num2 ))
echo $(( num1 / num2 ))
echo $(( num1 % num2 ))
```

Code Description:

- `$(( ))` performs arithmetic evaluation.
- This script calculates addition, subtraction, multiplication, division, and modulo for two integers.
- I have displayed results directly using `echo`.

Script 2: arithmetic_demo2.sh (Integer Arithmetic using `expr`)

```
#!/bin/bash

num1=30
```

```
num2=6
echo $(expr $num1 + $num2 )
echo $(expr $num1 - $num2 )
echo $(expr $num1 \* $num2 )
echo $(expr $num1 / $num2 )
echo $(expr $num1 % $num2 )
```

Code Description:

- **expr** is used to evaluate arithmetic expressions in Bash.
- I have escaped the multiplication operator ***** with *****.
- This method also handles integer operations.

Script 3: arithmetic_demo3.sh (Floating-Point Arithmetic using **bc**)

```
#!/bin/bash
```

```
num1=30.5
num2=6
```

```
echo "30.5+6" | bc
echo "30.5-6" | bc
echo "30.5*6" | bc
echo "30.5/6" | bc
echo "30.5%6" | bc
```

```
echo "scale=2;30.5/6" | bc
```

Code Description:

- **bc** is used for floating-point arithmetic in Bash.
- **scale=2** defines the precision for division.
- I have demonstrated addition, subtraction, multiplication, division, and modulo with decimals.

Script 4: arithmetic_demo4.sh (Advanced Floating-Point Arithmetic with **bc -l**)

```
#!/bin/bash
```

```
num1=30.5
num2=6
echo "$num1 + $num2" | bc
echo "$num1 - $num2" | bc
num=27
```

```
echo "scale=2;sqrt($num)" | bc -l
```

Code Description:

- `bc -l` invokes the math library for advanced operations like square root.
- I have calculated addition, subtraction, and square root with scale precision.
- Using `$num1` and `$num2` makes the script dynamic for different inputs.

Execution

After saving the scripts, I executed each one in the terminal using:

```
bash arithmetic_demo1.sh
```

```
bash arithmetic_demo2.sh
```

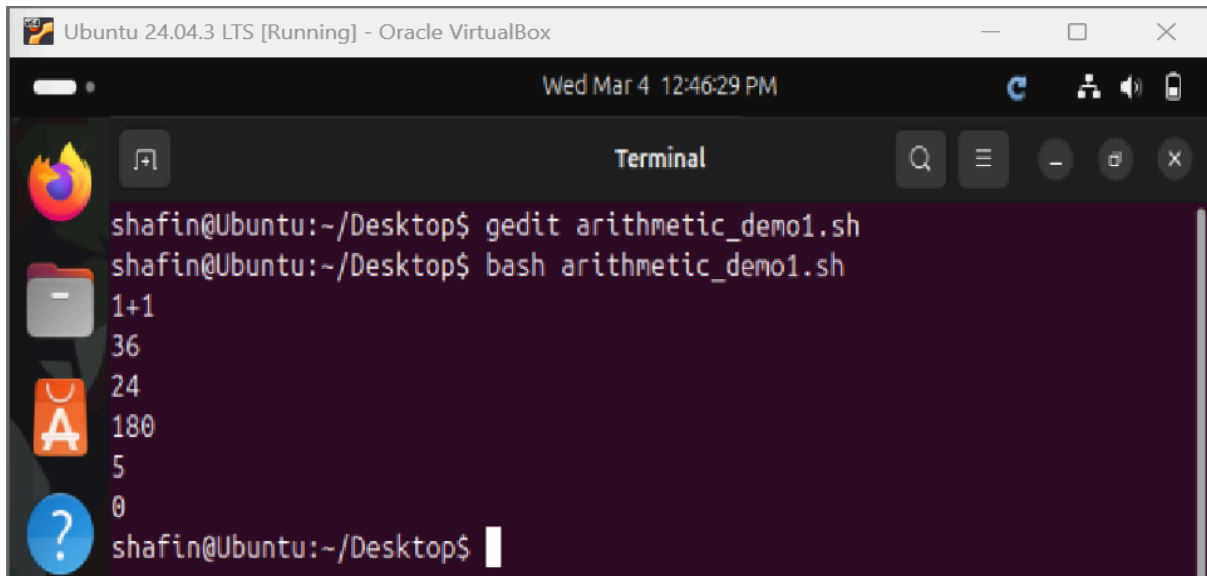
```
bash arithmetic_demo3.sh
```

```
bash arithmetic_demo4.sh
```

Output



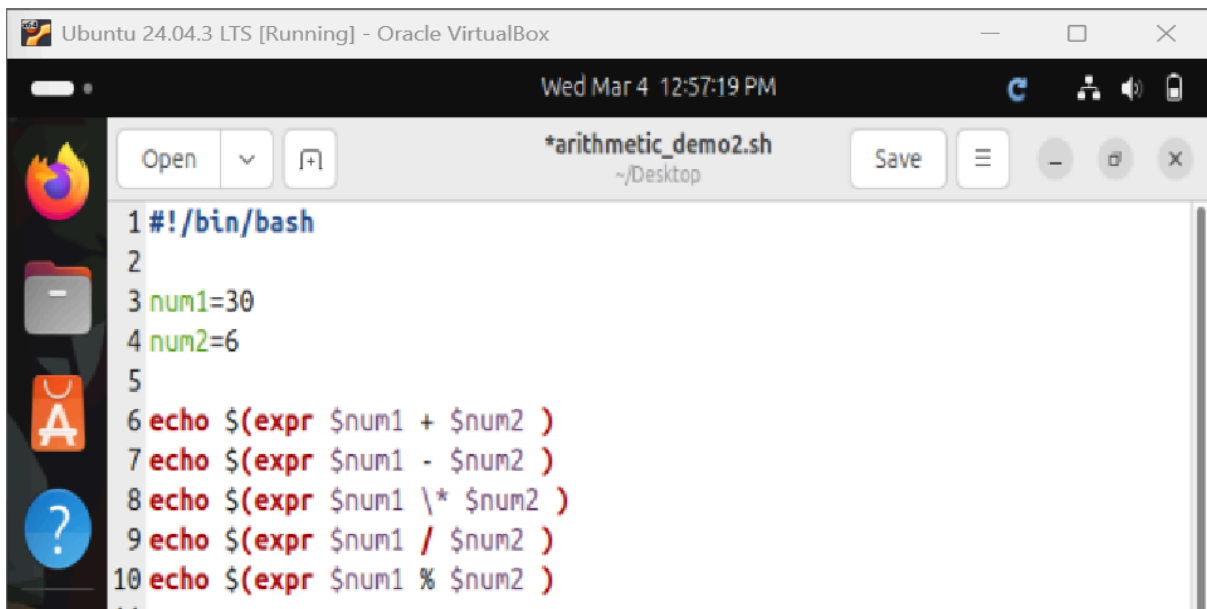
Figure 1: Text Editor Showing Script 1



The image shows a terminal window titled "Ubuntu 24.04.3 LTS [Running] - Oracle VirtualBox". The window has a dark theme. The terminal output shows the user "shafin" at the prompt "shafin@Ubuntu:~/Desktop\$". The user runs "gedit arithmetic_demo1.sh" and then "bash arithmetic_demo1.sh". The script outputs the following results:

```
shafin@Ubuntu:~/Desktop$ gedit arithmetic_demo1.sh
shafin@Ubuntu:~/Desktop$ bash arithmetic_demo1.sh
1+1
36
24
180
5
0
shafin@Ubuntu:~/Desktop$
```

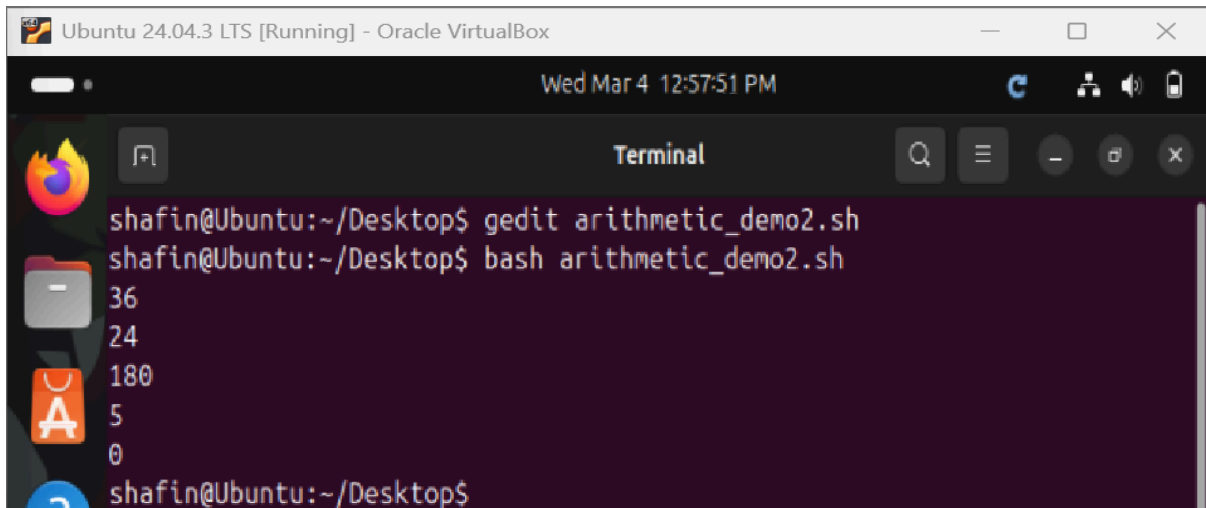
Figure 2: Terminal Output of Script 1



The image shows a text editor window titled "Ubuntu 24.04.3 LTS [Running] - Oracle VirtualBox". The window has a light theme. The editor is editing a file named "arithmetic_demo2.sh" located at "~/Desktop". The script content is as follows:

```
1 #!/bin/bash
2
3 num1=30
4 num2=6
5
6 echo $(expr $num1 + $num2 )
7 echo $(expr $num1 - $num2 )
8 echo $(expr $num1 \* $num2 )
9 echo $(expr $num1 / $num2 )
10 echo $(expr $num1 % $num2 )
11
```

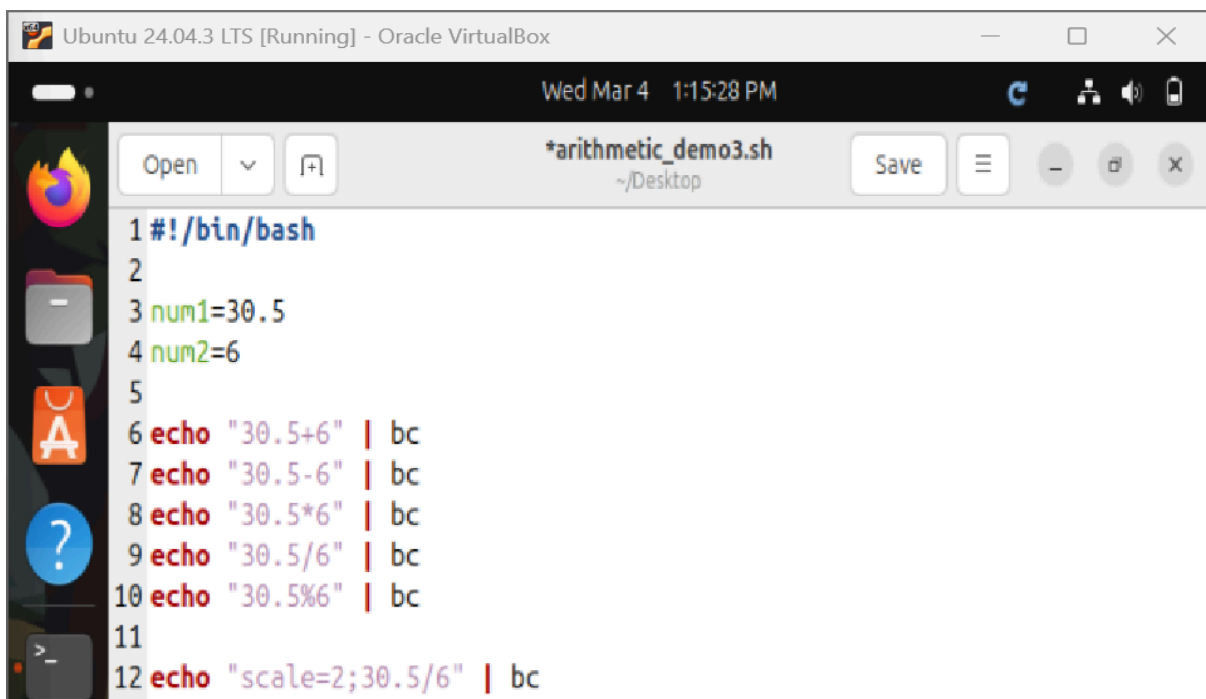
Figure 3: Text Editor Showing Script 2



The image shows a terminal window titled "Terminal" within an "Ubuntu 24.04.3 LTS [Running] - Oracle VirtualBox" environment. The window displays the following commands and output:

```
shafin@Ubuntu:~/Desktop$ gedit arithmetic_demo2.sh
shafin@Ubuntu:~/Desktop$ bash arithmetic_demo2.sh
36
24
180
5
0
shafin@Ubuntu:~/Desktop$
```

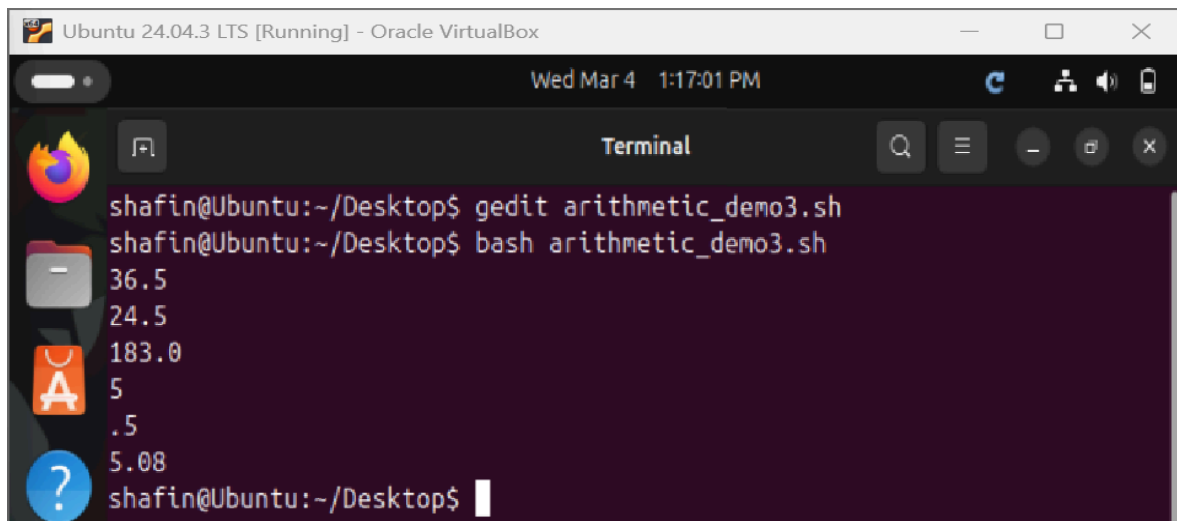
Figure 4: Terminal Output of Script 2



The image shows a text editor window titled "*arithmetic_demo3.sh" within an "Ubuntu 24.04.3 LTS [Running] - Oracle VirtualBox" environment. The window displays the following script content:

```
1 #!/bin/bash
2
3 num1=30.5
4 num2=6
5
6 echo "30.5+6" | bc
7 echo "30.5-6" | bc
8 echo "30.5*6" | bc
9 echo "30.5/6" | bc
10 echo "30.5%6" | bc
11
12 echo "scale=2;30.5/6" | bc
```

Figure 5: Text Editor Showing Script 3

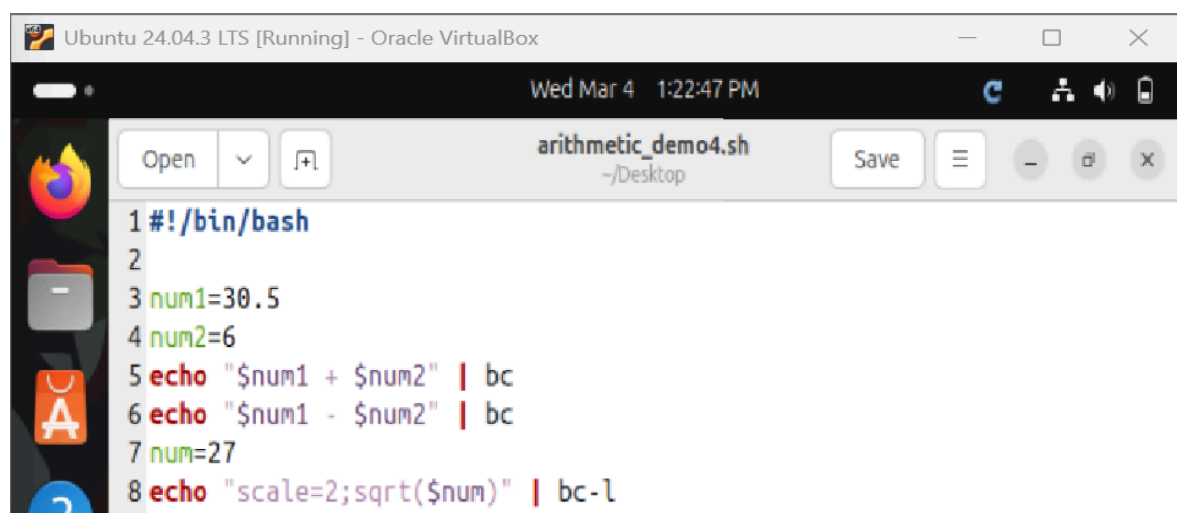


```

shafin@Ubuntu:~/Desktop$ gedit arithmetic_demo3.sh
shafin@Ubuntu:~/Desktop$ bash arithmetic_demo3.sh
36.5
24.5
183.0
5
.5
5.08
shafin@Ubuntu:~/Desktop$

```

Figure 6: Terminal Output of Script 3

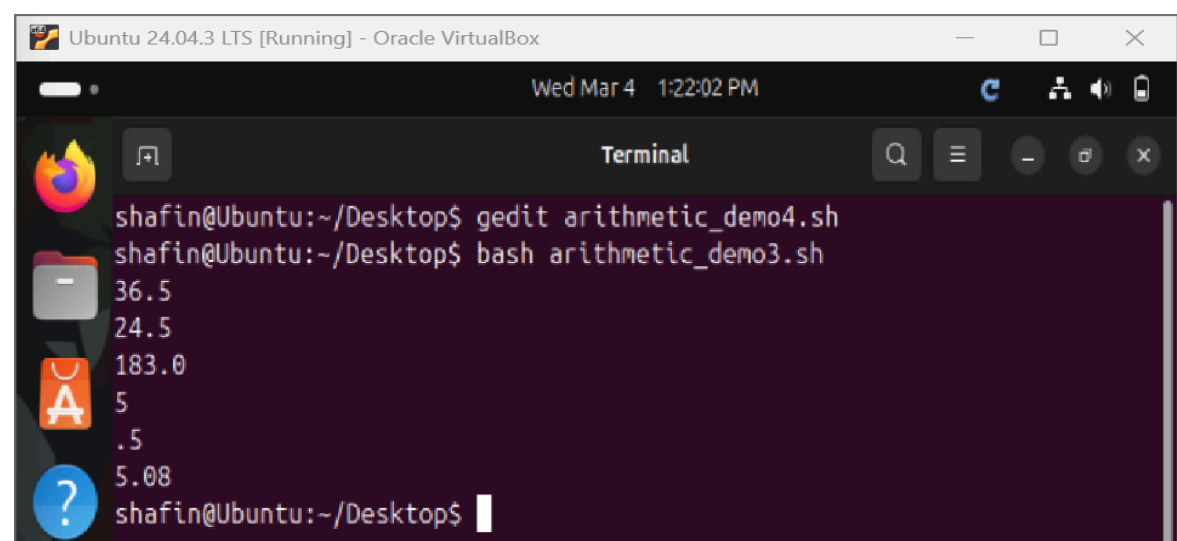


```

1#!/bin/bash
2
3num1=30.5
4num2=6
5echo "$num1 + $num2" | bc
6echo "$num1 - $num2" | bc
7num=27
8echo "scale=2;sqrt($num)" | bc-l

```

Figure 7: Text Editor Showing Script 4



```

shafin@Ubuntu:~/Desktop$ gedit arithmetic_demo4.sh
shafin@Ubuntu:~/Desktop$ bash arithmetic_demo3.sh
36.5
24.5
183.0
5
.5
5.08
shafin@Ubuntu:~/Desktop$

```

Figure 8: Terminal Output of Script 4

Discussion

Reflection

Through this experiment, I have learned multiple ways to perform arithmetic operations in Bash. I have understood:

- `$(( ))` and `expr` handle integer operations.
- `bc` is required for floating-point arithmetic.
- `bc -l` enables advanced mathematical operations like square root.
- Using `scale` allows control over decimal precision.

This experiment has strengthened my understanding of shell arithmetic and dynamic numeric computation.

Challenges Faced

Initially, I had to escape the `*` symbol when using `expr` to avoid syntax errors. I also ensured proper spacing for `bc` commands to prevent evaluation errors.

Conclusion

In this experiment, I have successfully implemented integer and floating-point arithmetic operations in Bash using `$(( ))`, `expr`, and `bc`. I have also applied the math library to compute square roots with precision.

These concepts have practical applications in automation scripts, calculations in shell programs, and mathematical processing in Linux environments.

Experiment No: 11

Experiment Name: Implementation of Functions in Bash

Script Name: functions_demo.sh

Introduction

Overview

In this experiment, I have implemented user-defined functions in a Bash shell script. The primary objective of this experiment has been to understand how functions are declared, defined, and executed in Bash scripting.

I have demonstrated how to pass arguments to a function and how to terminate script execution using a custom function.

Topics Covered

- Function declaration in Bash
- Passing arguments to functions
- Positional parameters (\$1, \$2, \$3)
- `exit` command
- Function calling

Environment / Platform

I have performed this experiment using Ubuntu Linux in the Bash shell environment.

I have used the **gedit** text editor to create and edit the script file and executed it using the terminal.

Task

Description

The task was to create a Bash script that:

1. Defines a function named `write` that prints three arguments.
2. Defines another function named `quit` that terminates the script.
3. Calls the `write` function with three arguments.
4. Prints a message after the function call.
5. Calls the `quit` function to exit the script.

Source Code

```
#!/bin/bash
function write() {
    echo $1 $2 $3
}
quit() {
    exit
}
write Tom and Jerry
echo "done"
quit
```

Code Description

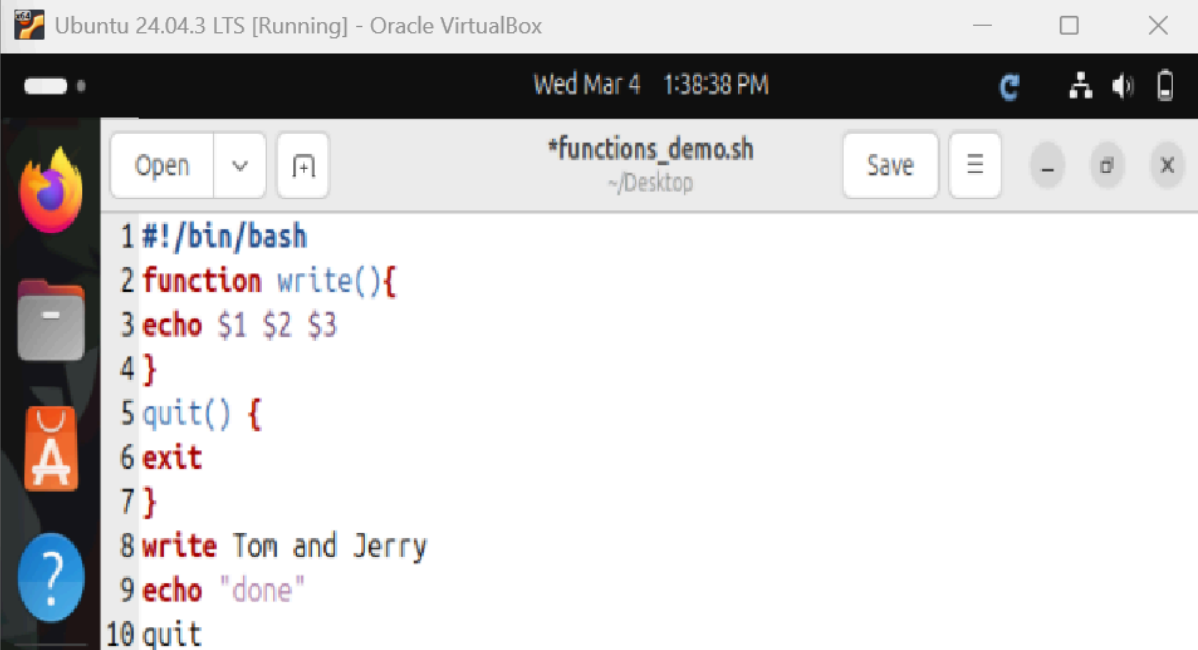
- `#!/bin/bash`
I have used this shebang line to specify that the script should run using the Bash interpreter.
- `function write()`
I have defined a function named `write`.
- `echo $1 $2 $3`
I have used positional parameters to display the first three arguments passed to the function.
- `quit()`
I have defined another function named `quit` that terminates the script.
- `exit`
I have used the `exit` command to stop script execution.
- `write Tom and Jerry`
I have called the function and passed three arguments: Tom, and, Jerry.
- `echo "done"`
I have printed a message after calling the function.
- `quit`
I have called the `quit` function to end the script.

Execution

After saving the file as `functions_demo.sh`, I executed it using:

```
bash functions_demo.sh
```

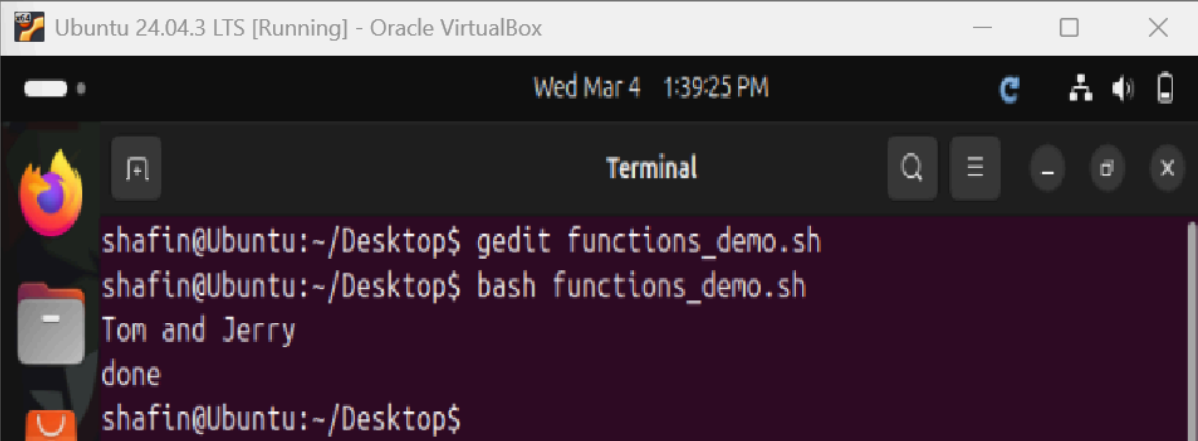
Output



The screenshot shows a window titled "Ubuntu 24.04.3 LTS [Running] - Oracle VirtualBox" with a timestamp of "Wed Mar 4 1:38:38 PM". The window contains a text editor (gedit) with the file name "*functions_demo.sh" and the path "~/Desktop". The editor displays the following code:

```
1#!/bin/bash
2function write(){
3echo $1 $2 $3
4}
5quit() {
6exit
7}
8write Tom and Jerry
9echo "done"
10quit
```

Figure 1: Text Editor (gedit) Showing Source Code



The screenshot shows a terminal window titled "Terminal" with a timestamp of "Wed Mar 4 1:39:25 PM". The terminal displays the following commands and output:

```
shafin@Ubuntu:~/Desktop$ gedit functions_demo.sh
shafin@Ubuntu:~/Desktop$ bash functions_demo.sh
Tom and Jerry
done
shafin@Ubuntu:~/Desktop$
```

Figure 2: Terminal Showing Execution Output

The script successfully displayed the function output and then terminated using the `quit` function.

Discussion

Reflection

Through this experiment, I have learned how to define and use functions in Bash scripting. I have understood how positional parameters work inside functions and how arguments are passed during function calls.

I have also learned how to use the `exit` command inside a function to terminate script execution.

Challenges Faced

Initially, I ensured correct syntax while defining functions and avoided using unnecessary spaces inside parentheses. After correcting minor formatting issues, the script executed successfully.

Conclusion

In this experiment, I have successfully implemented user-defined functions in Bash. I have demonstrated argument passing and script termination using functions.

This concept is important in shell scripting because functions improve code reusability, organization, and maintainability in larger automation scripts.

Experiment No: 12

Experiment Name: Local Variables in Bash Functions

Script Name: local_variable_demo.sh

Introduction

Overview

In this experiment, I have implemented local variables inside a Bash function. The primary objective of this experiment has been to understand the scope of variables in Bash scripting and how the `local` keyword restricts variable visibility within a function.

I have demonstrated the difference between local and global variables by observing their behavior inside and outside the function.

Topics Covered

- Function definition in Bash
 - Local variables using `local` keyword
 - Variable scope (local vs global)
 - Positional parameters
 - Echo command
-

Environment / Platform

I have performed this experiment using Ubuntu Linux in the Bash shell environment.

I have used the `gedit` text editor to write the script and executed it using the terminal.

Task

Description

The task was to:

1. Define a function named `write`.
 2. Declare a local variable inside the function using the `local` keyword.
 3. Print the value of the local variable inside the function.
 4. Attempt to access the same variable outside the function.
 5. Observe the behavior of variable scope.
-

Corrected Source Code

```
#!/bin/bash

function write() {
    local name=$1
    echo "The first name is $name"
}
name="Tom"
echo "The second name is $name"
write Jerry
echo "The third name is $name"
```

Code Description

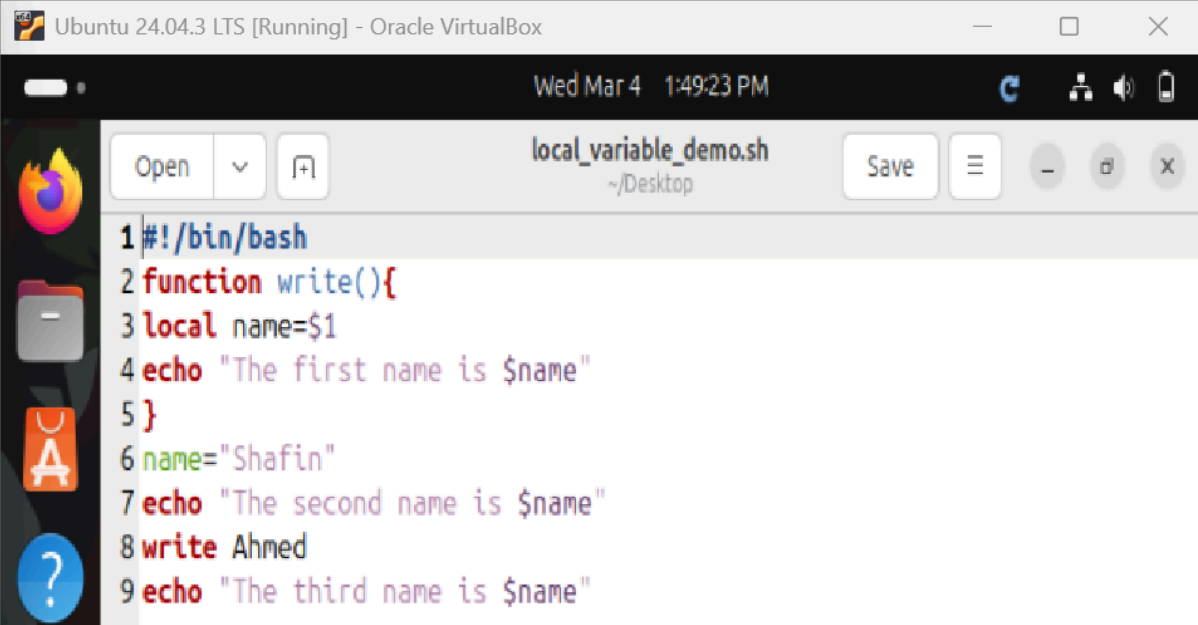
- `#!/bin/bash`
I have specified the Bash interpreter.
 - `function write()`
I have defined a function named `write`.
 - `local name=$1`
I have declared a local variable `name` inside the function. This variable is only accessible within the function.
 - `echo "The first name is $name"`
I have printed the local variable inside the function.
 - `name="Tom"`
I have declared a global variable outside the function.
 - `write Jerry`
I have called the function and passed "Jerry" as an argument.
 - The final `echo` statement shows that the global variable remains unchanged after the function execution.
-

Execution

After saving the script as `local_variable_demo.sh`, I executed it using:

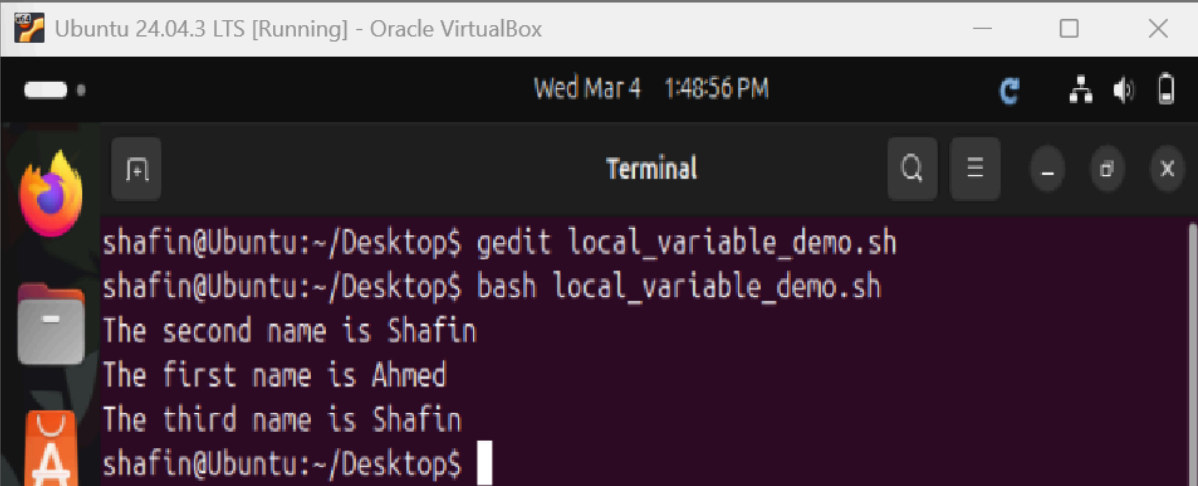
```
bash local_variable_demo.sh
```

Output



```
1#!/bin/bash
2function write(){
3local name=$1
4echo "The first name is $name"
5}
6name="Shafin"
7echo "The second name is $name"
8write Ahmed
9echo "The third name is $name"
```

Figure 1: Text Editor (gedit) Showing Source Code



```
shafin@Ubuntu:~/Desktop$ gedit local_variable_demo.sh
shafin@Ubuntu:~/Desktop$ bash local_variable_demo.sh
The second name is Shafin
The first name is Ahmed
The third name is Shafin
shafin@Ubuntu:~/Desktop$
```

Figure 2: Terminal Showing Execution Output

Discussion

Reflection

Through this experiment, I have learned how local variables work in Bash functions. I have understood that a variable declared with the `local` keyword is accessible only within the function block and does not affect the global variable with the same name.

I have also observed that the global variable retains its original value even after the function execution.

Challenges Faced

Initially, I corrected quotation marks and variable assignment syntax to avoid errors. After fixing these minor issues, the script executed successfully and demonstrated variable scope behavior correctly.

Conclusion

In this experiment, I have successfully implemented local variables in a Bash function. I have clearly observed the difference between local and global variable scopes.

This concept is essential for writing structured, maintainable, and error-free shell scripts, especially in larger automation programs.

Experiment No: 13

Experiment Name: Mini Project – Password Generator (Character Control)

Script Name: pass.sh

Introduction

Overview

In this mini project, I have developed a Password Generator using Bash scripting. The main objective of this project has been to generate a secure random password based on user-defined length.

I have implemented input validation to ensure that only numeric values are accepted. If the user enters a valid number, the script generates a password of the specified length. Otherwise, it displays an error message.

Objectives

- To generate random passwords using Bash
- To validate numeric user input
- To control password length dynamically
- To use `openssl` for secure random generation
- To implement conditional statements

Environment / Platform

I have performed this project using:

- Ubuntu Linux
- Bash Shell
- gedit text editor
- Terminal

Task Description

The task was to create a password generator script that:

1. Prompts the user to enter the desired password length.
 2. Validates whether the input is numeric.
 3. Generates a random password using `openssl`.
 4. Displays an error message if input is invalid.
-

Corrected Source Code

```
#!/bin/bash

echo "Enter the length of your password:"
read PASS_LENGTH

# Check if input is a valid number
if [[ $PASS_LENGTH =~ ^[0-9]+$ ]]; then
    # Generate password using openssl
    openssl rand -base64 48 | cut -c1-$PASS_LENGTH
else
    echo "Please enter a valid number."
fi
```

Code Explanation

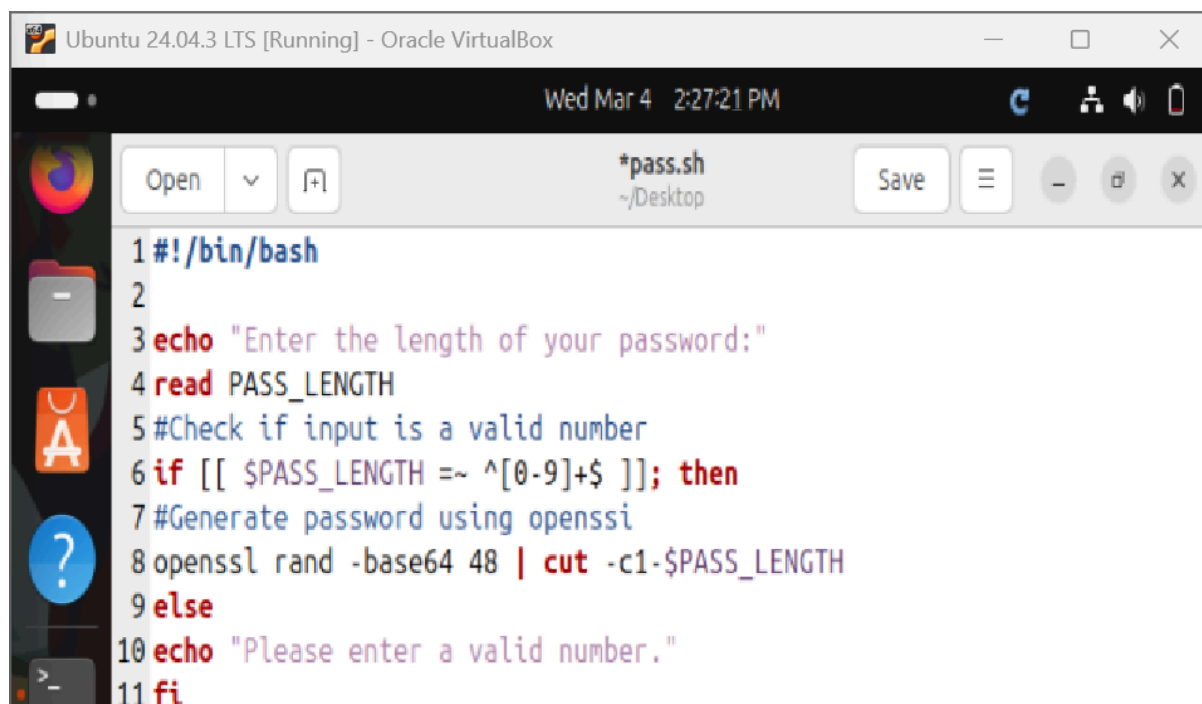
- `#!/bin/bash`
I have specified the Bash interpreter.
 - `read PASS_LENGTH`
I have taken user input for password length.
 - `[[ $PASS_LENGTH =~ ^[0-9]+$ ]]`
I have used a regular expression to ensure that only numeric values are accepted.
 - `openssl rand -base64 48`
I have generated a random string using OpenSSL.
 - `cut -c1-$PASS_LENGTH`
I have trimmed the random string to match the user-defined length.
 - `else` block
If the user enters invalid input, the script displays an error message.
-

Execution

After saving the file as `pass.sh`, I executed it using:

```
bash pass.sh
```

Output (Screenshots to Include)



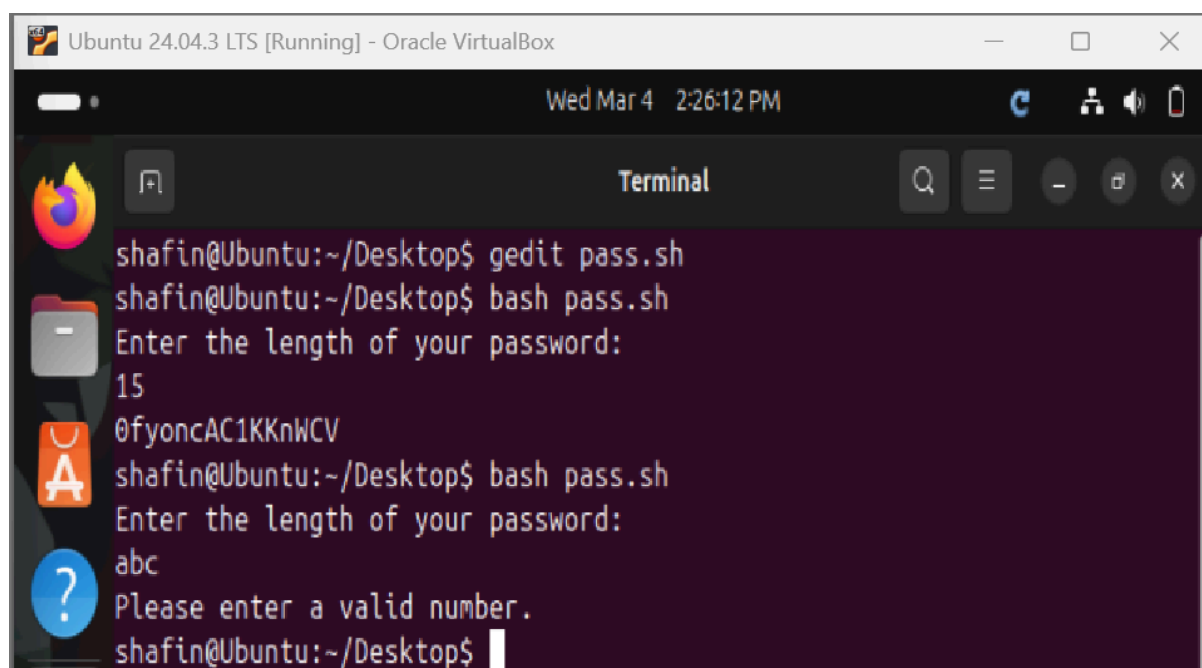
The screenshot shows a text editor window titled "Ubuntu 24.04.3 LTS [Running] - Oracle VirtualBox". The editor is displaying the source code for a script named "pass.sh" located at "~/Desktop". The code is as follows:

```

1 #!/bin/bash
2
3 echo "Enter the length of your password:"
4 read PASS_LENGTH
5 #Check if input is a valid number
6 if [[ $PASS_LENGTH =~ ^[0-9]+$ ]]; then
7 #Generate password using openssl
8 openssl rand -base64 48 | cut -c1-$PASS_LENGTH
9 else
10 echo "Please enter a valid number."
11 fi

```

Fig 01: Source Code in Text Editor



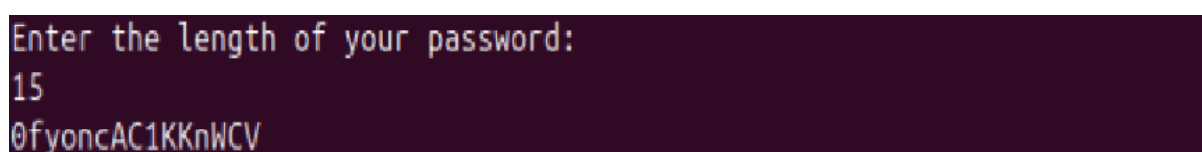
The screenshot shows a terminal window titled "Ubuntu 24.04.3 LTS [Running] - Oracle VirtualBox". The terminal displays the following commands and output:

```

shafin@Ubuntu:~/Desktop$ gedit pass.sh
shafin@Ubuntu:~/Desktop$ bash pass.sh
Enter the length of your password:
15
0fyoncAC1KKnWCV
shafin@Ubuntu:~/Desktop$ bash pass.sh
Enter the length of your password:
abc
Please enter a valid number.
shafin@Ubuntu:~/Desktop$

```

Fig 02: Showing Output in Terminal based on Valid Input & Output



This is a close-up screenshot of the terminal output from the previous figure, showing the generated password:

```

Enter the length of your password:
15
0fyoncAC1KKnWCV

```

Fig 03: Output showing generated password

Discussion

Reflection

Through this mini project, I have learned how to generate secure random passwords using Bash scripting. I have understood how to validate user input using regular expressions and how to manipulate command output using pipes.

I have also gained practical experience using `openssl` for secure password generation.

Challenges Faced

Initially, I ensured proper syntax for the regular expression and correct usage of the `cut` command. After correcting minor formatting issues, the script worked successfully.

Conclusion

In this mini project, I have successfully developed a password generator with character length control. The script validates user input and generates a secure random password dynamically.

This project demonstrates practical application of Bash scripting concepts such as conditional statements, regular expressions, command piping, and secure random generation.
